

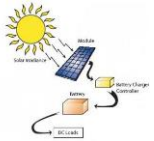
8.3 微机系统串行扩展原理

8.3.1 - 串行接口技术综述

8.3.2 - 串行移位和SPI总线

8.3.3 - I^2C总线

8.3.4 - 1-wire总线



8.3.1串行接口技术综述

特点:占用IO口极少

串行接口与UART的区别:

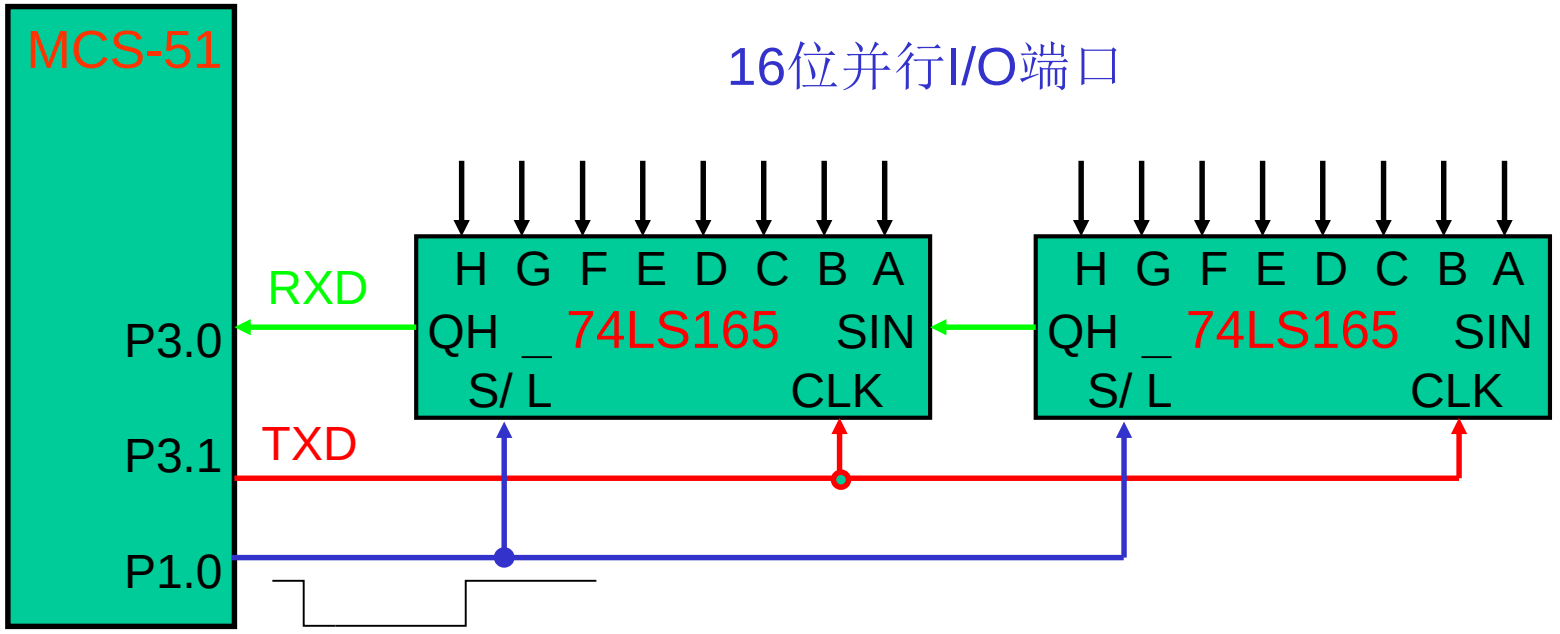
UART一般用于较长距离MCU与MCU间通信，速率较低；

串行接口一般用于一块线路板内部MCU与外围芯片间的通信，速率较高；

UART通信只定义了单个字符的数据格式；串行接口则定义了完整的地址读写格式。

8.3.2 - 串行移位和SPI总线

使用2片74LS165构成的16位输入扩展口

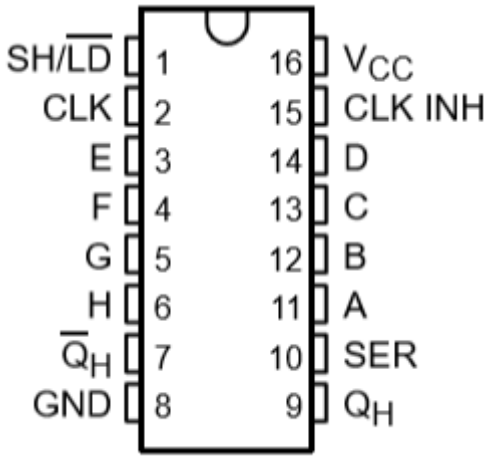


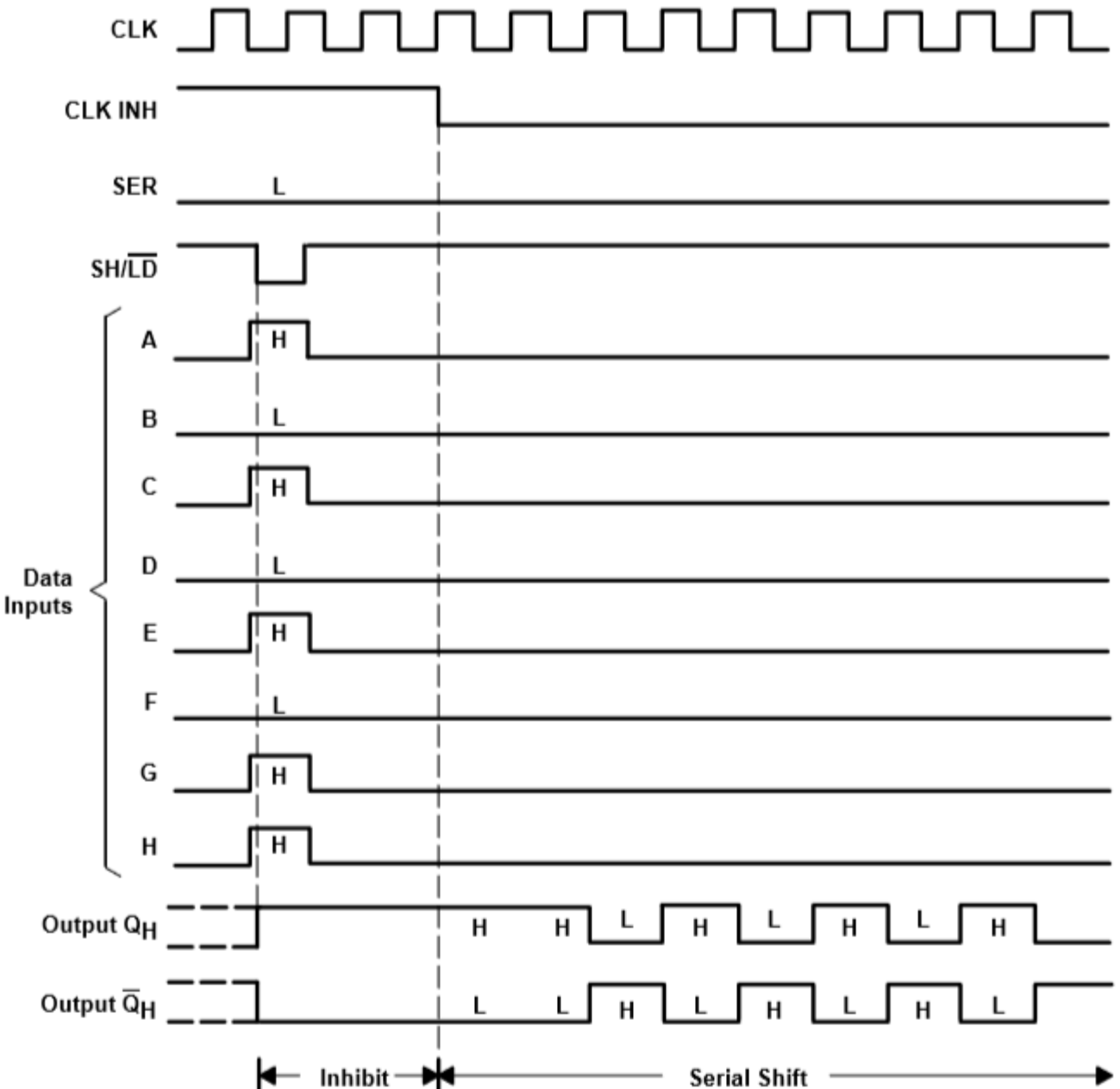
74LS165:

并入串出移位寄存器

- A～H:8位并行数据输入端；
- SIN: 串行数据输入端；
- QH: 串行数据输出端；
- $S/\overline{L}$: =0时锁存并行数据，=1时允许串行移位。

74LS165


SH/$\overline{LD}$ 1
CLK 2
E 3
F 4
G 5
H 6
$\overline{Q}_H$ 7
GND 8
16 V_{CC}
15 CLK INH
14 D
13 C
12 B
11 A
10 SER
9 Q_H


CLK
CLK INH
SER
L
SH/$\overline{LD}$
Data Inputs
A
H
B
L
C
H
D
L
E
H
F
L
G
H
H
H
Output Q_H
H H L H L H L H
Output $\overline{Q}_H$
L L H L H L H L
Inhibit
Serial Shift

编程举例：读入20个字节的数据并送入内部RAM单元

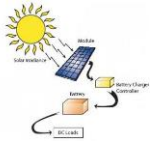
```
        MOV   R7,#10              ;字节数20，循环10次
        MOV   R0,#20H             ;内部数据区指针R0赋初值
        MOV   SCON,#00000000B     ;串口设定模式0
        SETB  RI                  ;关闭接收数据
        SETB  REN                 ;接收允许
LOOP:   CLR   P1.0                ;并行置入数据
        SETB  P1.0                ;允许串行移位
        CLR   RI                  ;启动接收
        JNB   RI,$                ;等待接收结束
        MOV   A,SBUF              ;将接收的数据送A
        MOV   @R0,A               ;数据送内存
        INC   R0                  ;修改数据区指针R0
        CLR   RI                  ; 启动接收
        JNB   RI,$                ;等待接收结束
        MOV   A,SBUF              ;将接收的数据送A
        MOV   @R0,A               ;数据送内存
        INC   R0                  ;修改数据区指针R0
        DJNZ R7,LOOP              ;是否完成10次循环
        RET
```

锁存数据

编程举例：读入20个字节的数据并送入内部RAM单元

```
        MOV   R7,#20          ;字节数20送计数器R7
        MOV   R0,#20H         ;内部数据区指针R0赋初值
        SETB  F0              ;设置“是否锁存”标志
RCV0:   CLR   P1.0            ;并行置入数据
        SETB  P1.0            ;允许串行移位
RCV1:   MOV   SCON,#00010000B ;设定模式0并启动接收(REN=1)
LOOP:   JNB   RI,$            ;等待接收结束
        CLR   RI              ; 清RI准备下一次接收
        MOV   A,SBUF          ;将接收的数据送A
        MOV   @R0,A           ;数据送内存
        INC   R0              ;修改数据区指针R0
        CPL   F0              ;标志取反
        JB    F0,RCV2         ;接收偶次后重新置入数据
        DEC   R7              ;接收奇次时要再接收一帧
        SJMP  RCV1            ; 转RCV1接收第二帧数据
RCV2:   DJNZ  R7,RCV0         ;是否完成20个字节的输入
        RET
```

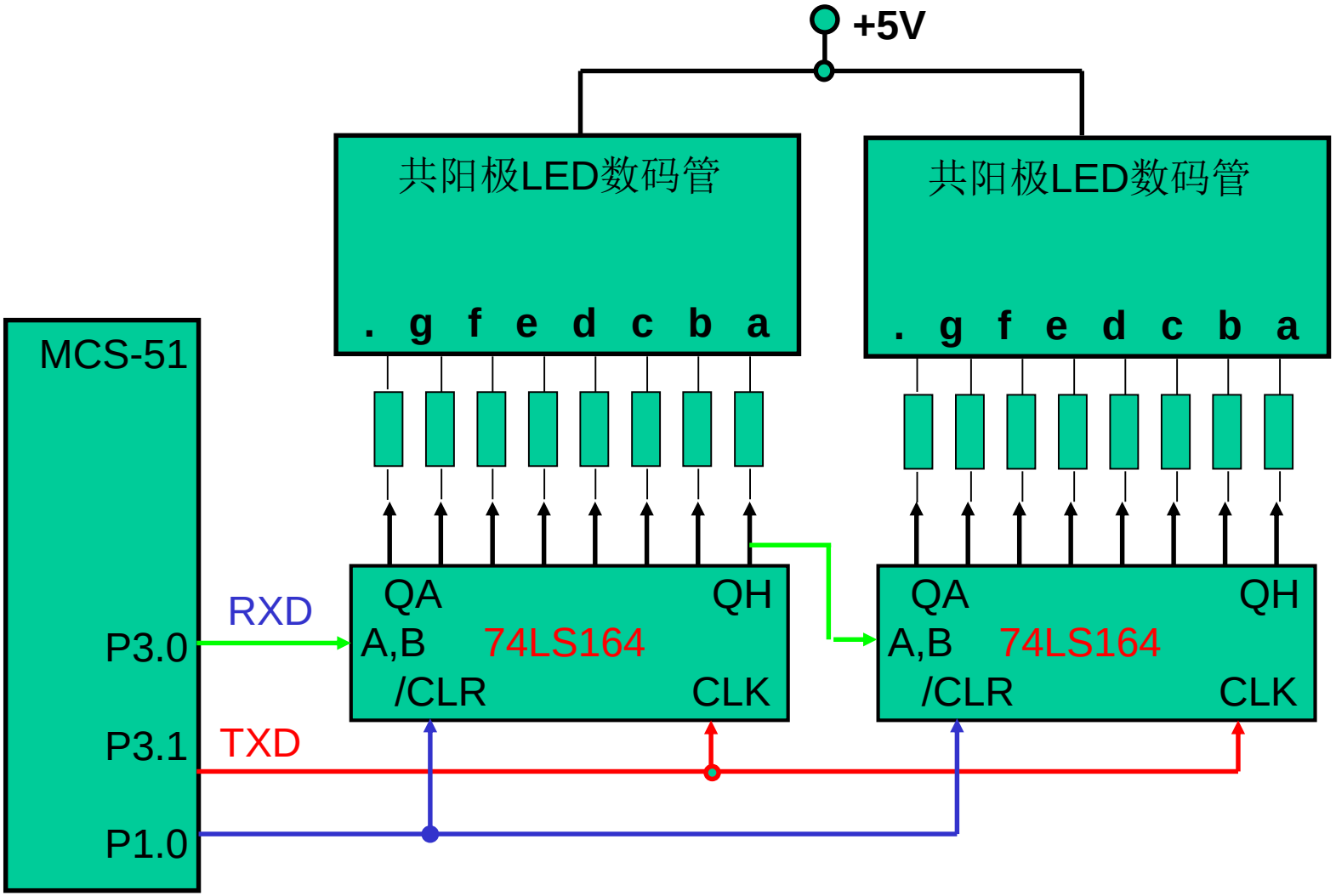
【注意】：F0的作用和程序根据F0的状态转移到RCV1、RCV0的区别。



直接I0口操作编程

```
        MOV   R7,#10          ;读字节数20，实际循环10次，送计数器R7
        MOV   R0,#20H         ;内部数据区指针R0赋初值
        SETB  P3.0
        SETB  P3.1
        SETB  P1.0
RCV0:   CLR   P1.0            ;并行置入数据
        SETB  P1.0            ;允许串行移位
        MOV   R1,#8
LOOP1:
        CLR   P3.1            ;CLK
        SETB  P3.1            ;CLK
        MOV   C,P3.0
        RLC   A
        DJNZ  R1,LOOP1
        MOV   @R0,A           ;数据送内存
        INC   R0
        MOV   R1,#8
LOOP2:
        CLR   P3.1            ;CLK
        SETB  P3.1            ;CLK
        MOV   C,P3.0
        RLC   A
        DJNZ  R1,LOOP2
        MOV   @R0,A           ;数据送内存
        INC   R0
        DJNZ R7,RCV0          ;是否完成20个字节的输入
        RET
```

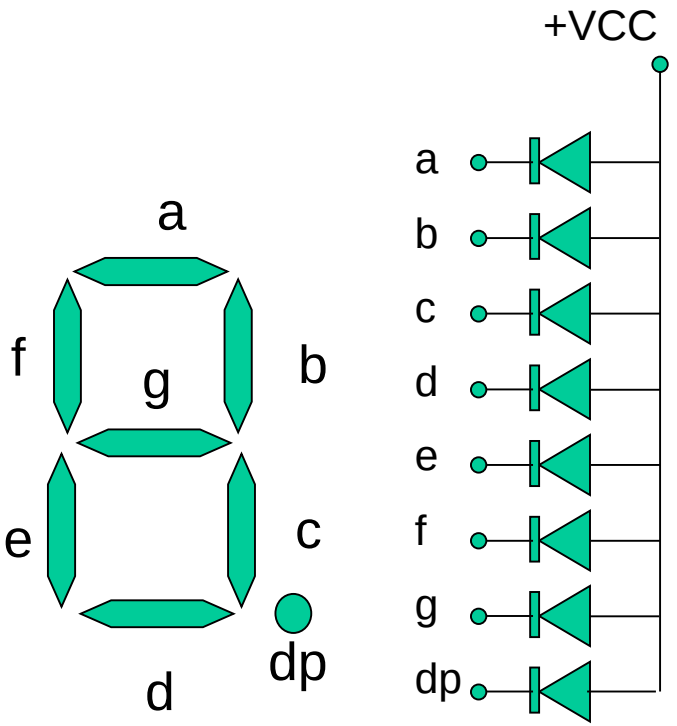
使用2片74LS164构成的16位输出扩展口

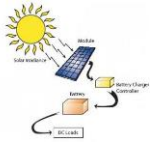


编程：利用静态数码管显示电路，显示RAM中的数据

- 数据存RAM的20H开始的单元，以BCD码的形式存储；
- 显示前必须利用查表程序将BCD码转换为共阳极的字符码。

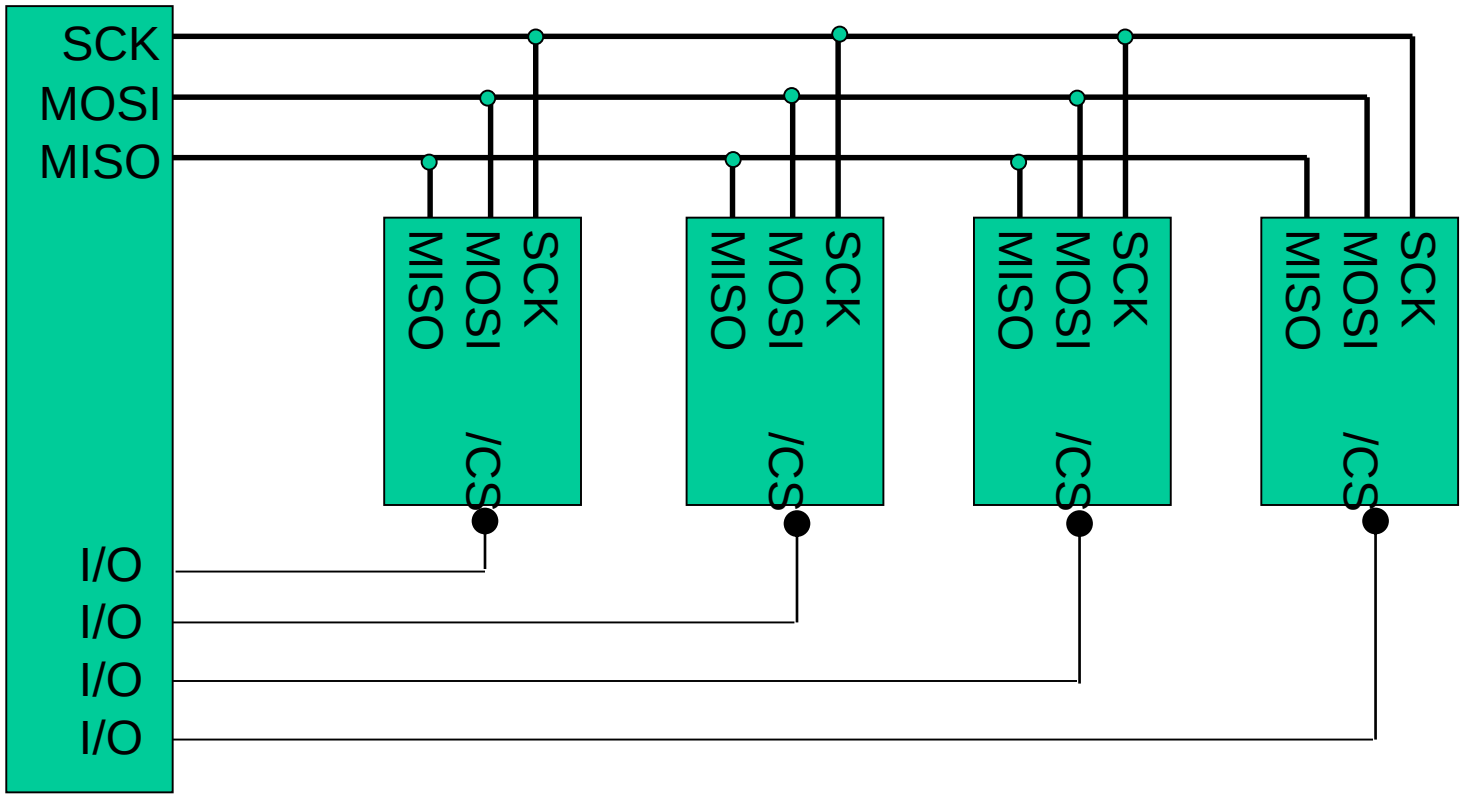
显示字型	dp ,g,f,e,d,c,b, a	字符码
0	1 1 0 0 0 0 0 0	C0H
1	1 1 1 1 1 0 0 1	F9H
2	1 0 1 0 0 1 0 0	A4H
3	1 0 1 1 0 0 0 0	B0H
4	1 0 0 1 1 0 0 1	99H
5	1 0 0 1 0 0 1 0	92H
6	1 0 0 0 0 0 1 0	82H
7	1 1 1 1 1 0 0 0	F8H
8	1 0 0 0 0 0 0 0	80H





```
        ORG  2000H
DISP:   MOV  R7,#02H          ;设置显示位数
        MOV  R0,#20H          ;设置数据区指针
        MOV  SCON,#00H        ;设串行口为模式0
        MOV  DPTR, # TABLE    ;设置字符表指针
DISP0:  MOV  A , @R0          ;取数据到A
        MOVC  A,@A+DPTR       ;从表中取与A对应的字符到A
        MOV  SBUF,A           ;字符向外传送
LOOP:   JNB   TI,$            ;等待传送结束
        CLR  TI               ;清除标志TI
        INC  R0               ;指向下一个数据
        DJNZ  R7,DISP0        ;未完时转DISP0继续
        RET
TABLE:  DB  0C0H,0F9H,0A4H,0B0H,99H      ;0,1, 2,3,4的字符码
        DB  92H,82H,0F8H,80H,98H         ;5,6,7,8,9的字符码
        END
```

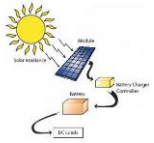
（二）SPI总线扩展系统



SCK：同步移位脉冲。

MOSI：主机输出、从机输入。

MISO：主机输入从机输出。



SPI总线工作原理

- 美国MOTOROLA公司推出的一种同步串行接口。
- SPI使用4条信号线：

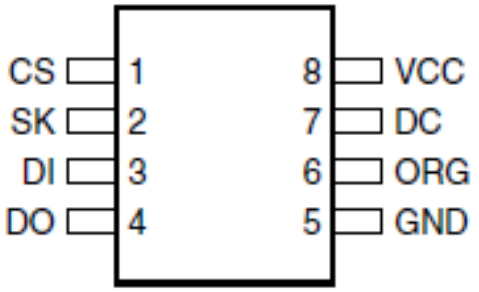
  ①同步时钟线SCK；　　②主机输入/从机接收线；

  ③主机输出/从机输入线；④从机片选线/CS（低电平）。
- SPI总线为“单主机系统”，主机为单片机，从机为外围芯片。主机通过/CS线来指定、寻找从机进行数据交换。
- 如果外围器件是单一功能的输入或输出，则连接可简化为3线系统。
- MCS-51单片机虽然没有SPI接口，但完全可以通过软件模拟SPI的时序完成SPI操作，但必须注意两点：①先传送D7位；②同步脉冲的有效边缘以外围器件要求为准。

SPI接口EEPROM芯片 93C46

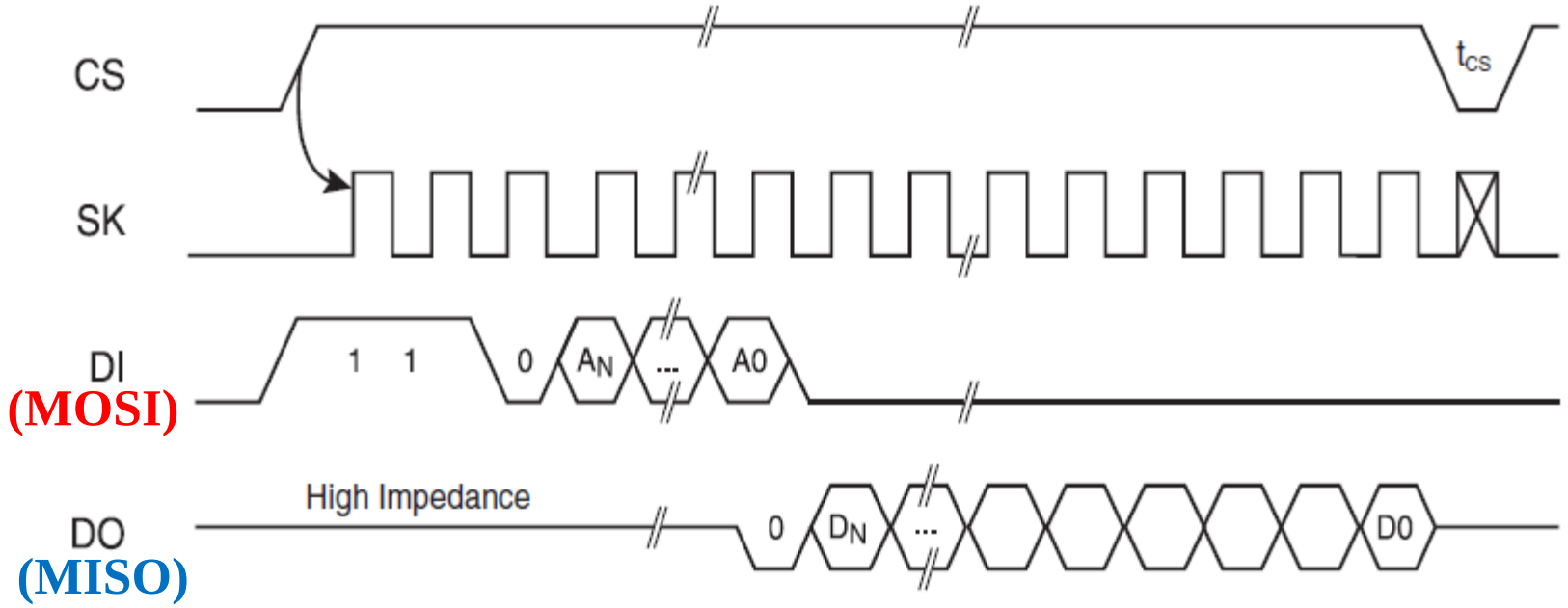
Features

- Low-voltage and Standard-voltage Operation
  - 2.7 (V_{CC} = 2.7V to 5.5V)
  - 1.8 (V_{CC} = 1.8V to 5.5V)
- User-selectable Internal Organization
  - 1K: 128 x 8 or 64 x 16
- Three-wire Serial Interface
- 2 MHz Clock Rate (5V)
- Self-timed Write Cycle (10 ms max)
- High Reliability
  - Endurance: 1 Million Write Cycles
  - Data Retention: 100 Years



SPI接口EEPROM芯片 93C46

READ Timing



I/O	AT93C46 (1K)	
	x 8	x 16
A_N	A_6	A_5
D_N	D_7	D_{15}

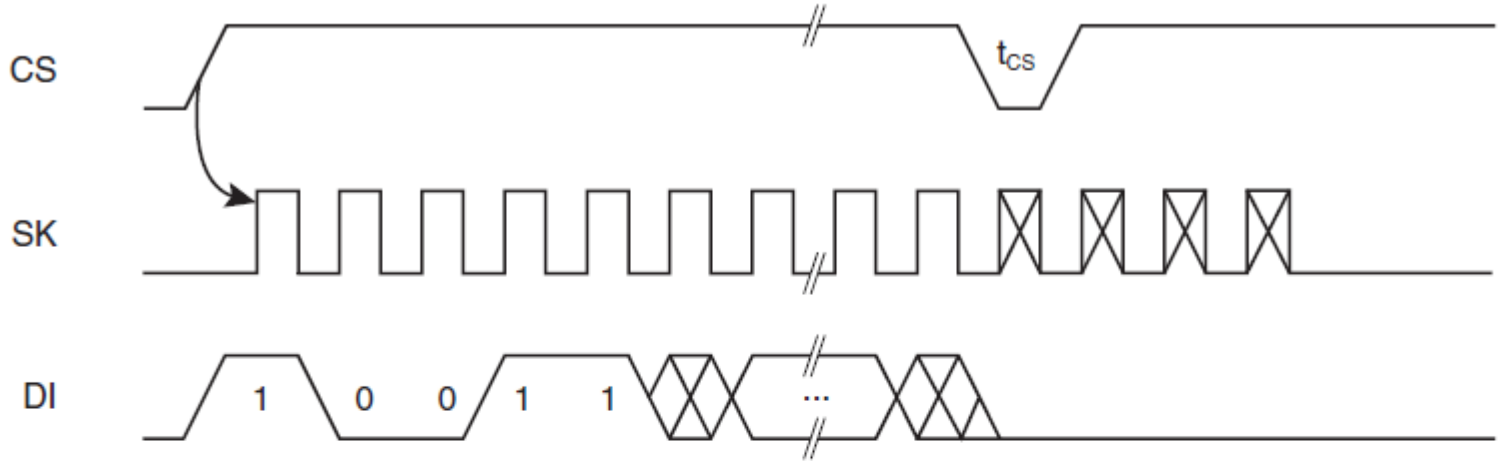
93C46 指令集

Instruction Set for the AT93C46

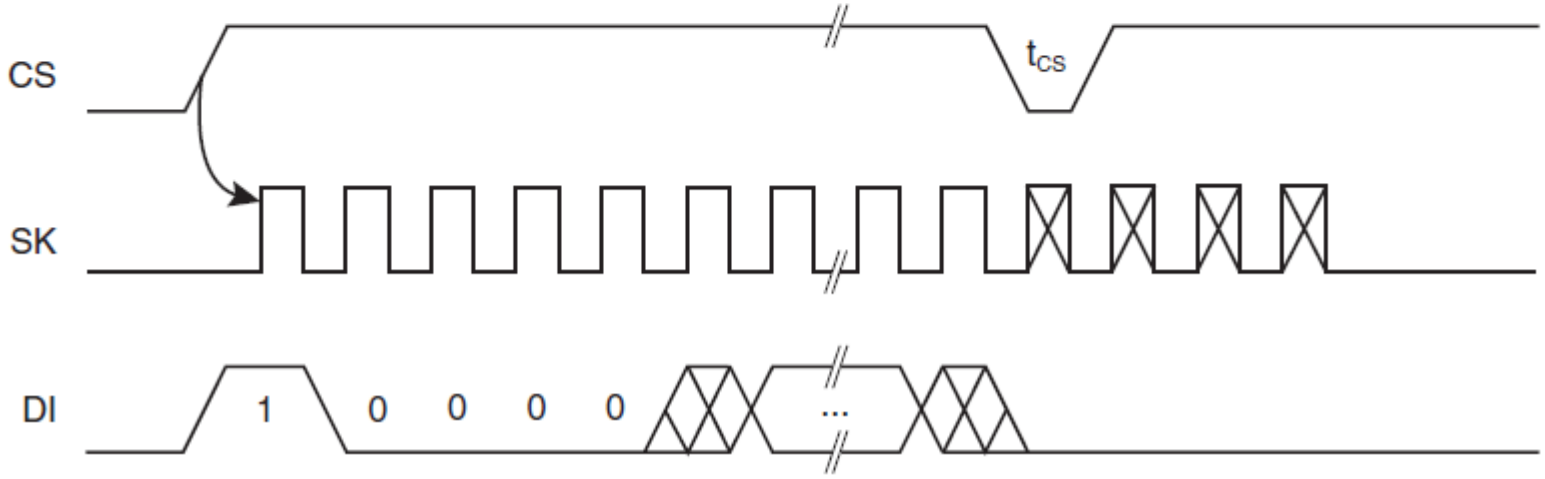
Instruction	SB	Op Code	Address		Data		Comments
			x 8	x 16	x 8	x 16	
READ	1	10	$A_6 - A_0$	$A_5 - A_0$			Reads data stored in memory, at specified address
EWEN	1	00	11XXXXX	11XXXX			Write enable must precede all programming modes
ERASE	1	11	$A_6 - A_0$	$A_5 - A_0$			Erases memory location $A_n - A_0$
WRITE	1	01	$A_6 - A_0$	$A_5 - A_0$	$D_7 - D_0$	$D_{15} - D_0$	Writes memory location $A_n - A_0$
ERAL	1	00	10XXXXX	10XXXX			Erases all memory locations. Valid only at V_{CC} = 4.5V to 5.5V
WRAL	1	00	01XXXXX	01XXXX	$D_7 - D_0$	$D_{15} - D_0$	Writes all memory locations. Valid only at V_{CC} = 4.5V to 5.5V
EWDS	1	00	00XXXXX	00XXXX			Disables all programming instructions

Note: The Xs in the address field represent *DON'T CARE* values and must be clocked.

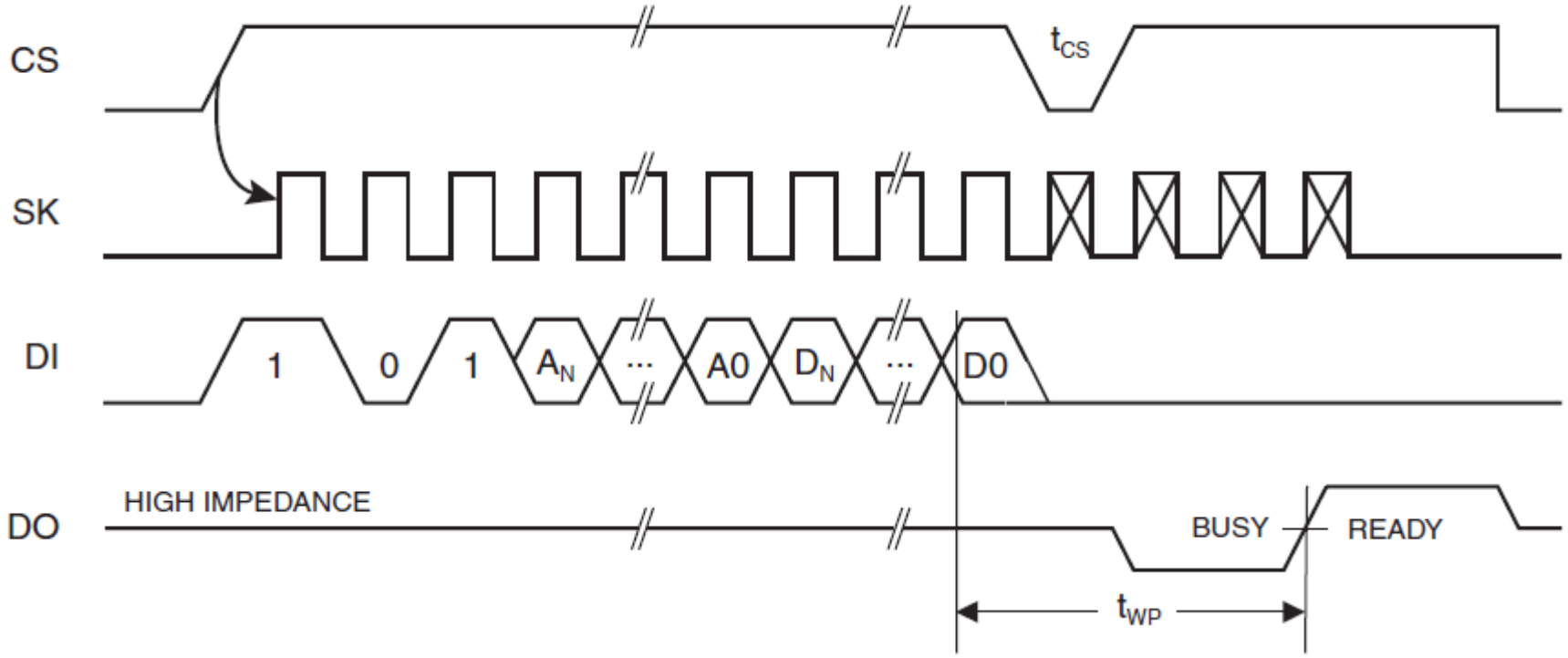
EWEN Timing



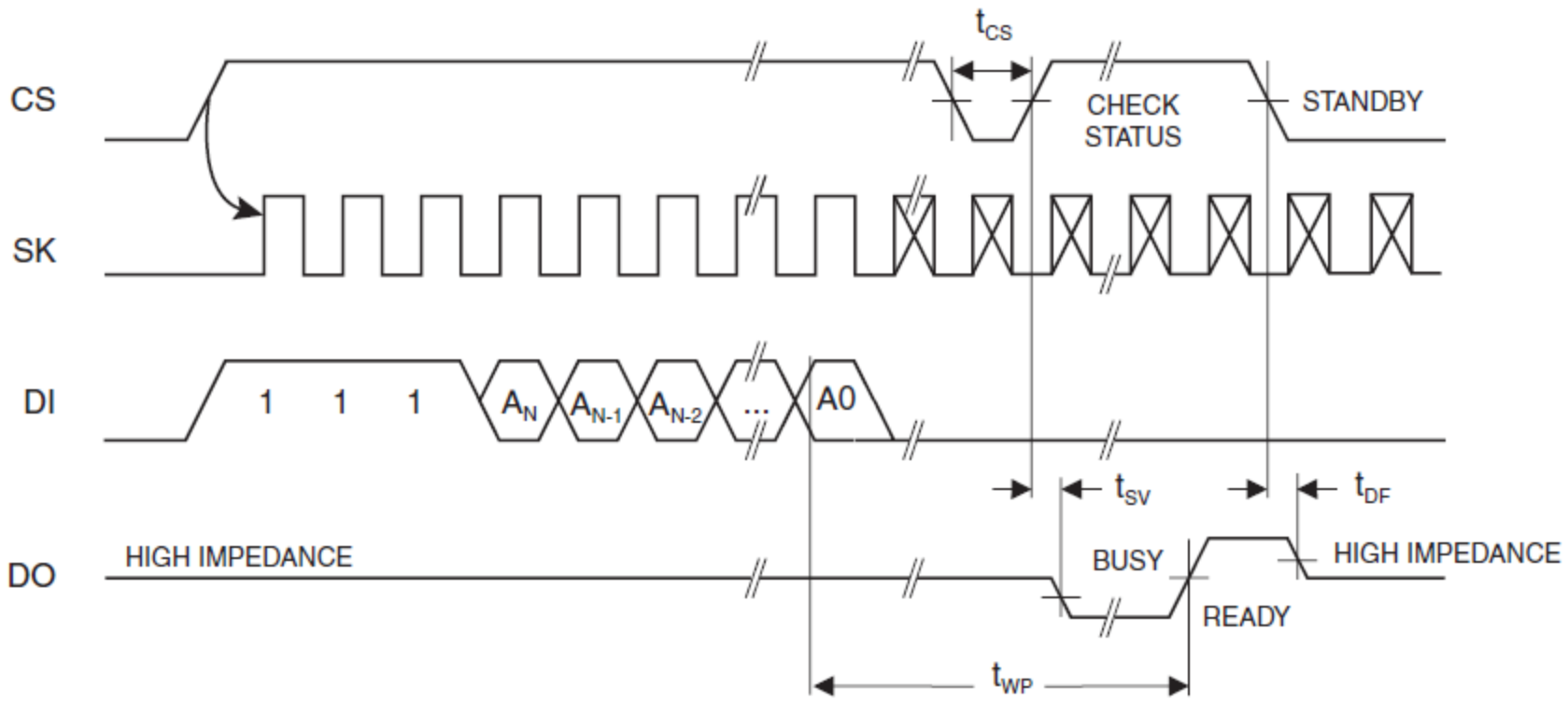
EWDS Timing



WRITE Timing



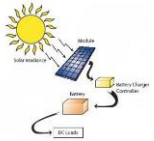
ERASE Timing



93C46程序设计

读指令子程序：

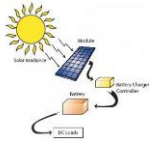
```
;;输入:R_ADDR—93C46地址
;;输出:R_DATA
READ:
        SETB    CS
        MOV     A,#0C0H          ;;发送数据
        MOV     R0,#3            ;;发送位数
        LCALL   SEND_DATA
        MOV     A,R_ADDR         ;;发送数据
        RL      A
        MOV     R0,#7            ;;7位地址
        LCALL   SEND_DATA
        LCALL   READ_BYTE
        MOV     R_DATA,A
        LCALL   STOP
        RET
```



93C46程序设计

连续发R0个位：

```
;;INPUT:  A--数据
;;          R0--有效位个数
;;
SEND_DATA:
        RLC     A
        MOV     DI,C
        SETB    SK
        NOP
        NOP
        CLR     SK
        NOP
        DJNZ    R0,SEND_DATA
        RET
```



93C46程序设计

读一个字节：

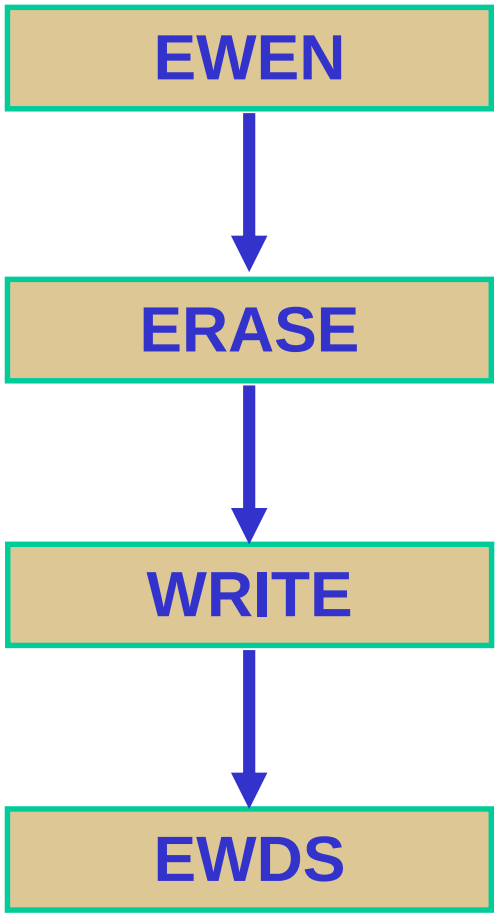
```
;;INPUT:  无
;; INFLUENCED:  R0
;;OUTPUT:  A—数据
READ_BYTE:
        MOV     R0,#8
REC_LOOP:
        SETB    SK
        NOP
        NOP
        MOV     C,DO
        RLC     A
        CLR     SK
        NOP
        DJNZ    R0,REC_LOOP
        RET
```

93C46程序设计

写指令子程序：

```
;;输入:W_ADDR—93C46地址
;;      W_DATA—数据
WRITE:
      SETB   CS
      MOV    A,#0A0H              ;;写命令字
      MOV    R0,#3
      LCALL  SEND_DATA
      MOV    A,W_ADDR             ;;写地址
      RLC    A                    ;;7位地址右对齐
      MOV    R0,#7
      LCALL  SEND_DATA
      MOV    A,W_DATA             ;;写数据
      MOV    R0,#8
      LCALL  SEND_DATA
      LCALL  STOP
      SETB   CS
      JNB    DO,$
      CLR    CS
      RET
```

93C46写数据的一般流程

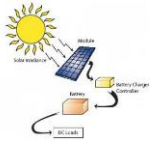


解除写保护

擦除原来的数据

写入新数据

开启写保护



Cortex-M4的SPI外设

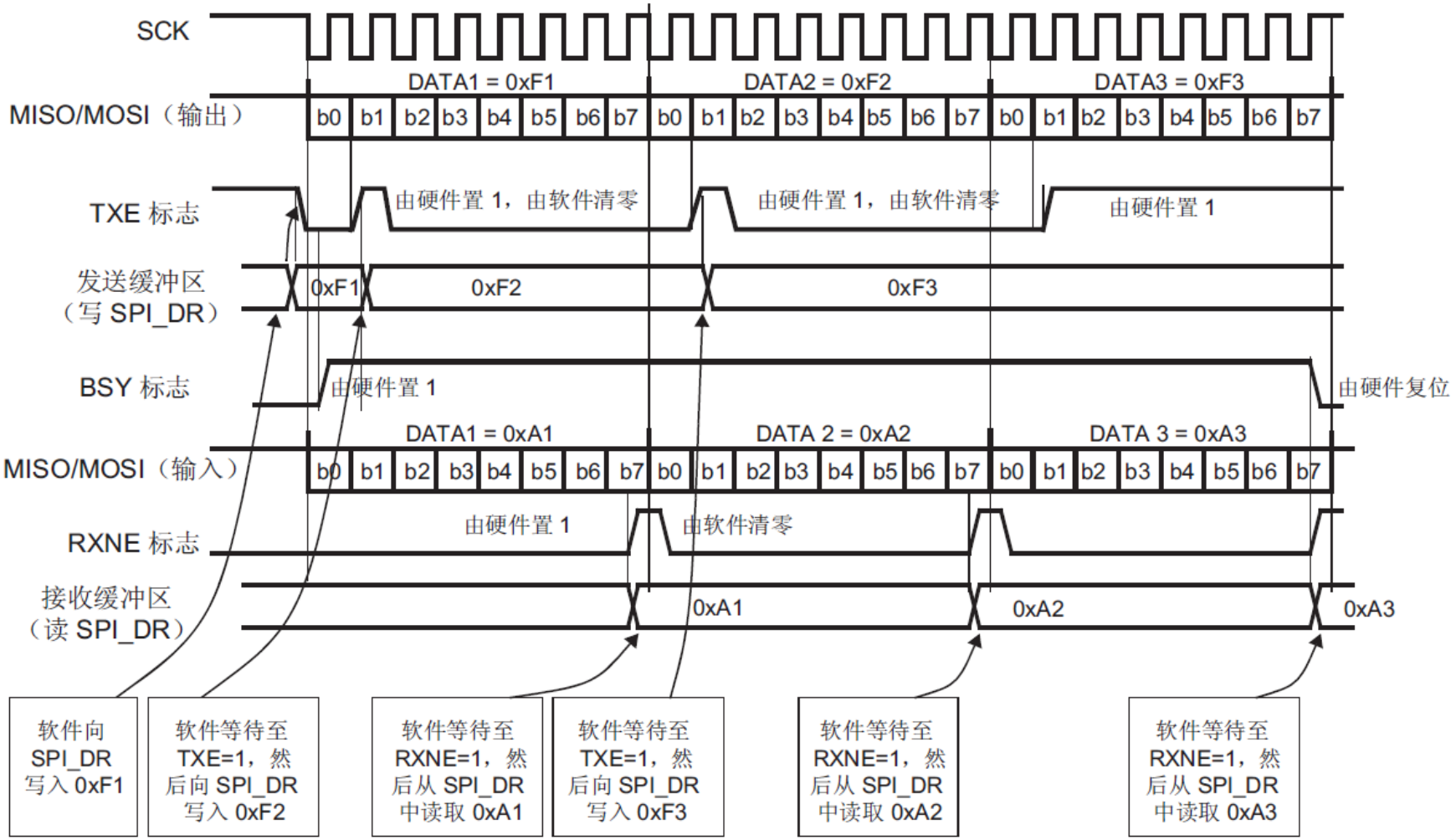
SPI 外设特性

- 基于三条线的全双工同步传输
- 基于双线的单工同步传输，其中一条可作为双向数据线
- 8 位或 16 位传输帧格式选择
- 主模式或从模式操作
- 多主模式功能
- 可编程的时钟极性和相位
- 可编程的数据顺序，最先移位 MSB 或 LSB
- 可触发中断的专用发送和接收标志

Cortex-M4的SPI外设

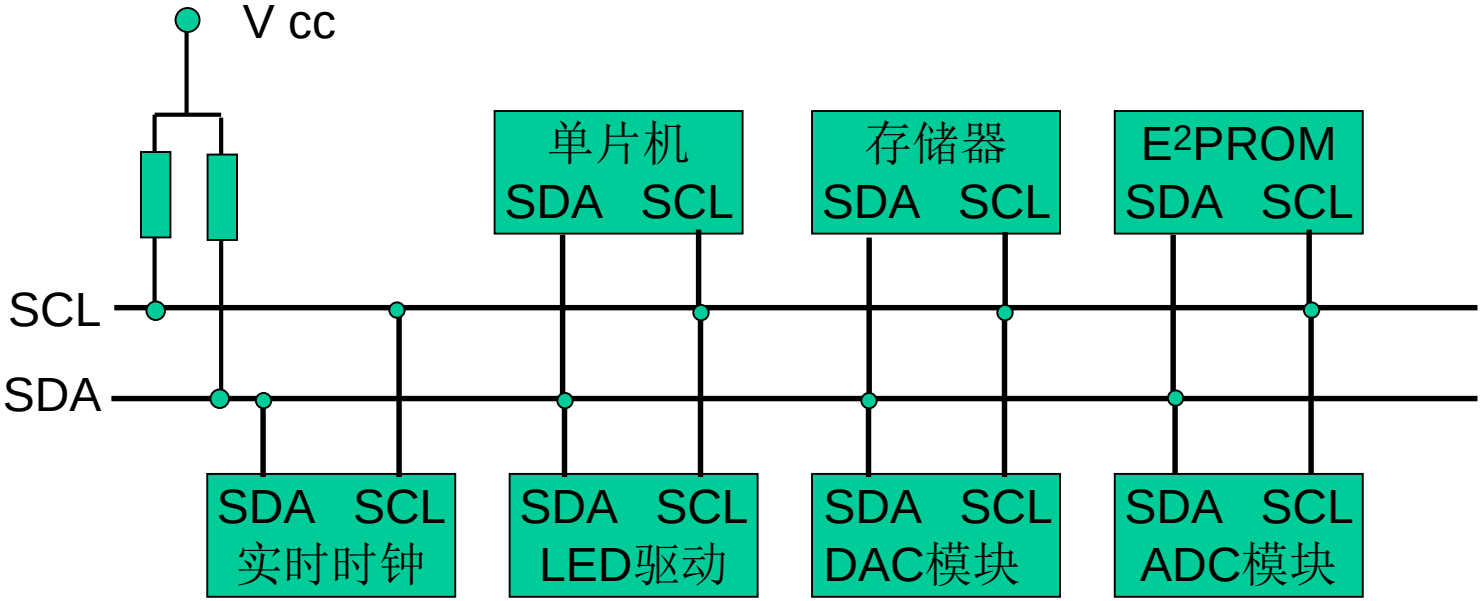
SPI通信时序示例

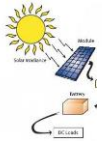
CPOL=1、CPHA=1 时的主模式示例



ai17343

8.3.3 I²C总线

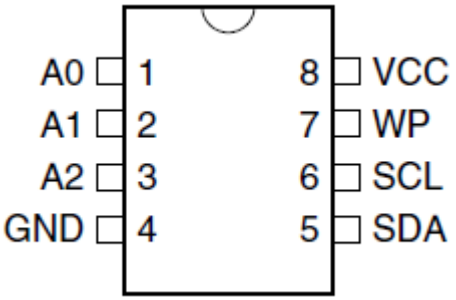

V cc
单片机
SDA SCL
存储器
SDA SCL
E²PROM
SDA SCL
SCL
SDA
SDA SCL
实时时钟
SDA SCL
LED驱动
SDA SCL
DAC模块
SDA SCL
ADC模块



I²C总线工作原理

- I²C总线采用两线制：数据线SDA、时钟线SCL。
- I²C总线为同步总线，数据的传送与时钟信号完全同步。
- I²C总线采用主从方式，由主控器寻址从器件，启动总线，产生时钟脉冲。
- 所有与SDA、SCL总线相连的器件（单片机和外围器件）都应当具备I²C总线接口。
- I²C总线接口器件在出厂时按照芯片的型号都具有唯一的8位地址（4位内部硬件地址、3位引脚定义的外部地址和1位方向地址）。

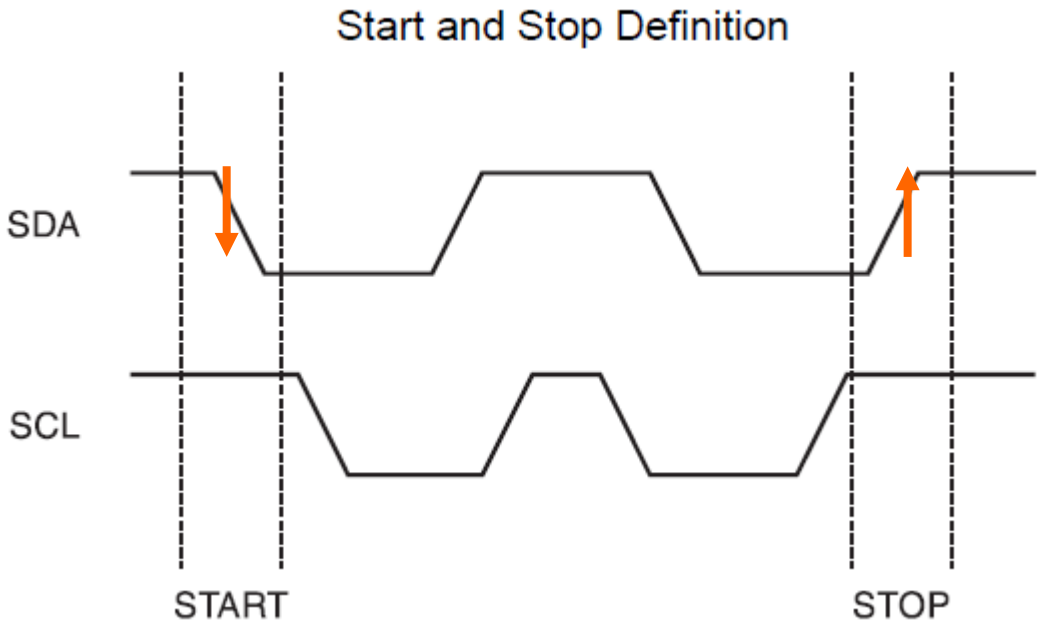
I²C总线EEPROM 24C01



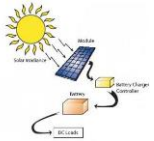
Features

- Low-voltage and Standard-voltage Operation
  - 2.7 (V_{CC} = 2.7V to 5.5V)
  - 1.8 (V_{CC} = 1.8V to 5.5V)
- Internally Organized 128 x 8 (1K), 256 x 8 (2K), 512 x 8 (4K), 1024 x 8 (8K) or 2048 x 8 (16K)
- Two-wire Serial Interface
- Schmitt Trigger, Filtered Inputs for Noise Suppression
- Bidirectional Data Transfer Protocol
- 100 kHz (1.8V) and 400 kHz (2.7V, 5V) Compatibility
- Write Protect Pin for Hardware Data Protection
- 8-byte Page (1K, 2K), 16-byte Page (4K, 8K, 16K) Write Modes
- Partial Page Writes Allowed
- Self-timed Write Cycle (5 ms max)
- High-reliability
  - Endurance: 1 Million Write Cycles
  - Data Retention: 100 Years

I²C总线EEPROM 24C01



I²C时序



I²C总线EEPROM 24C01

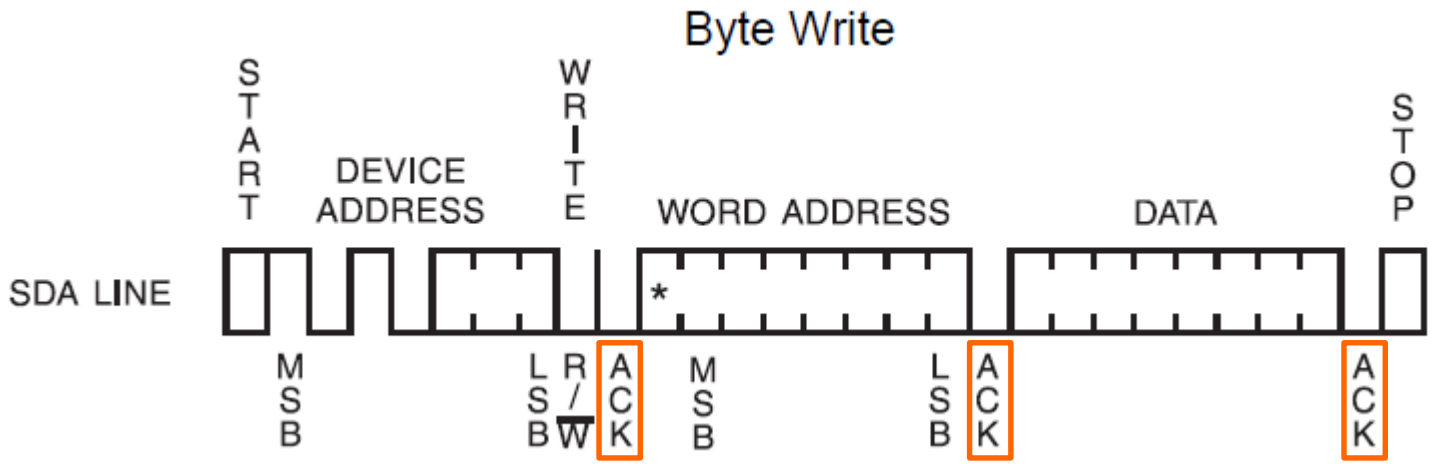
根据I²C时序Start和Stop定义可知：

在传输数据时，当SCL高电平，SDA不能变化。

因SCL由主机控制，所以：

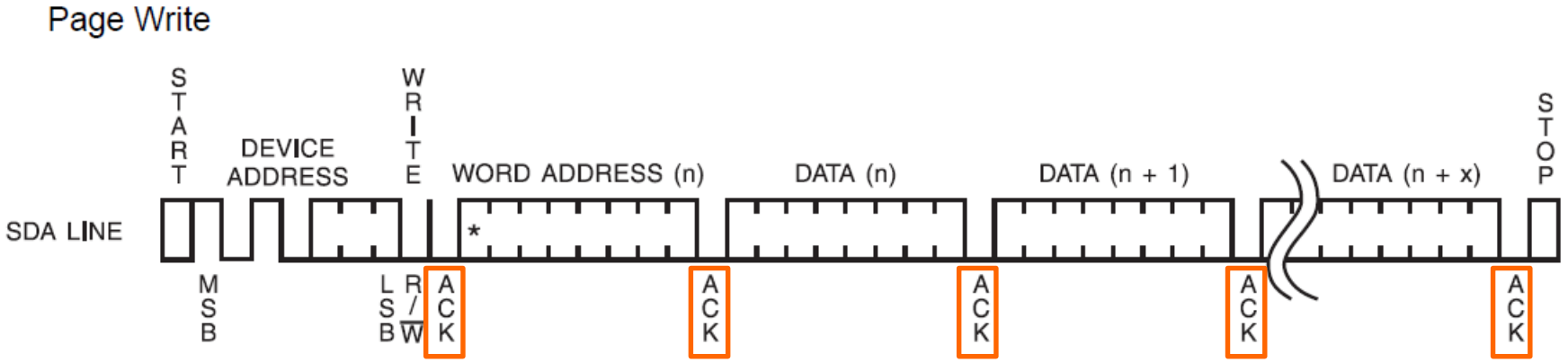
主机在SCL低电平时输出数据至SDA；

在SCL高电平时读取SDA数据。

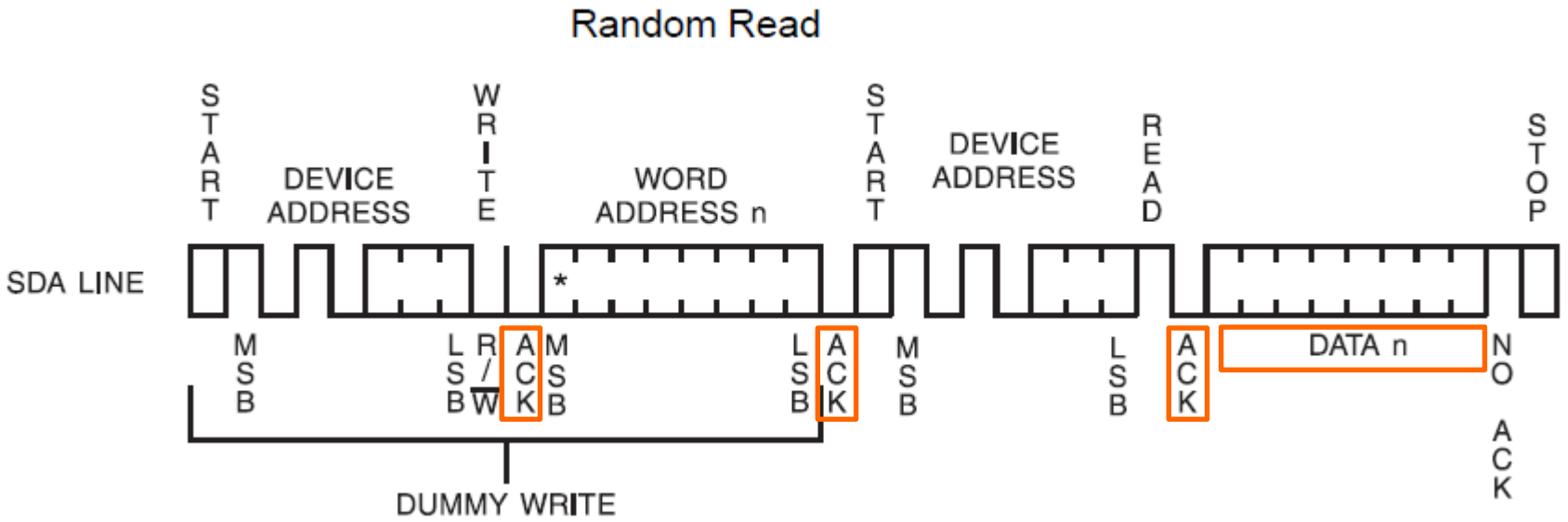


字节写时序

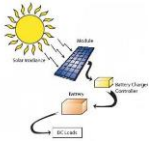
从机发出



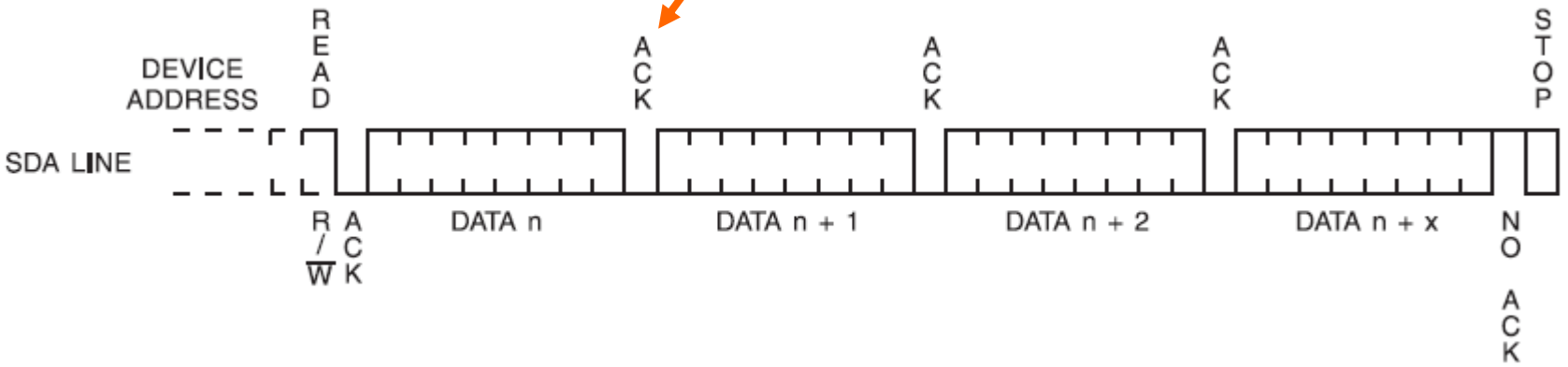
连续写时序


Random Read
START
DEVICE ADDRESS
WRITE
WORD ADDRESS n
START
DEVICE ADDRESS
READ
STOP
SDA LINE
*
MSB
LSB
R/W
ACK
MSB
LSB
ACK
MSB
LSB
ACK
DATA n
NO ACK
DUMMY WRITE

字节读时序



主机发出



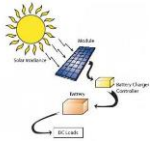
字节连续读时序

芯片的寻址

AT24C01A/02/04

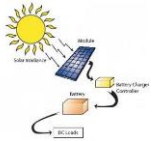
Device Address

1K/2K	1	0	1	0	A_2	A_1	A_0	R/W
	MSB							LSB
4K	1	0	1	0	A_2	A_1	P0	R/W
8K	1	0	1	0	A_2	P1	P0	R/W
16K	1	0	1	0	P2	P1	P0	R/W



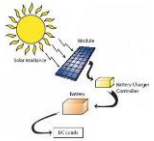
- ;;名称:START
- ;;描述:启动IIC总线子程序---发送IIC起始条件
- ;;**********
  ****

```
START:
        SETB    SDA             ;发送起始条件的数据信号
        NOP
        SETB    SCL             ;发送起始条件的时钟信号
        NOP                     ;起始条件建立时间大于4us
        NOP
        NOP
        NOP
        NOP
        CLR     SDA             ;发送起始信号
        NOP                     ;起始条件锁定时间大于4us
        NOP
        NOP
        NOP
        NOP
        CLR     SCL             ;钳住IIC总线,准备发送或接收数据
        RET
```



- ;;名称:STOP
- ;;描述:停止IIC总线子程序---发送IIC总线停止条件
- ;;**********
  ****

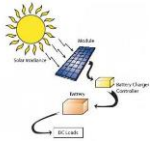
```
STOP:
        CLR     SDA             ;发送停止条件的数据信号
        NOP
        SETB    SCL             ;发送停止条件的时钟信号
        NOP                     ;起始条件建立时间大于4us
        NOP
        NOP
        NOP
        NOP
        SETB    SDA             ;发送IIC总线停止信号
        NOP
        NOP
        NOP
        NOP
        RET
```



- ;;接收应答
- ;;==========

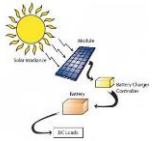
==========

```
ACKN:
        SETB        SDA
        NOP
        NOP
        SETB        SCL
        NOP
        JB          SDA,   $
        NOP
        NOP
        CLR         SCL
        RET
```

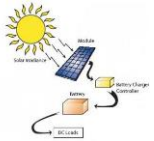


- ;;发送应答
- ;;==========

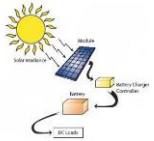
```
ACKS:
        CLR    SDA
        NOP
        NOP
        NOP
        SETB   SCL
        NOP
        NOP
        NOP
        NOP
        CLR    SCL
        RET
```



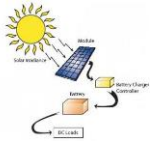
```
;;名称: RBYTE
;;功能: READ ONE BYTE DATA FROM 24C16
;;输入: (无)
;;输出: A
;;参数说明:
;;                    A:接收数据(1字节)
;;--------------------------------------------------------------------
RBYTE:
        SETB    SDA
        MOV     R0,     #8
XRB1:
        SETB    SCL
        NOP
        NOP
        NOP
        MOV     C,      SDA
        CLR     SCL
        RLC     A
        NOP
        NOP
        DJNZ    R0,     XRB1
        RET
```



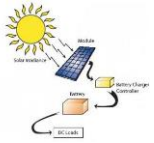
```
;;名称: WBYTE
;;功能: WRITE ONE BYTE DATA TO 24C16
;;输入: A
;;输出: (无)
;;参数说明:
;;                A:发送数据(1字节)
WBYTE:
        MOV     R0,#8
XWB1:
        RLC     A
        NOP
        NOP
        MOV     SDA,C
        NOP
        SETB    SCL
        NOP
        NOP
        NOP
        NOP
        NOP
        CLR     SCL
        DJNZ    R0,XWB1
        RET
```



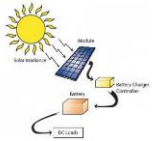
- ;;名称: W24C
- ;;功能: WRITE DATA TO 24C16
- ;;输入: SADD,R1,R2
- ;;输出: (无)
- ;;---
- ;;调用: START,WBYTE,ACKN,STOP
- ;;---
- ;;影响: A,R1,R2
- ;;堆栈: 4
- ;;---
- ;;参数说明:
- ;;　　SADDH,SADDL:24C16 ADDRESS
- ;;　　R2:WRITE NUMBER
- ;;　　R1:RAM ADDRESS POINTER
- ;;---
- ;;注意:
- ;;---



```
W24C:
        LCALL  START              ;I²C起始信号
        MOV    A,    SADDH
        RL     A
        ORL    A,    #0A0H
        LCALL  WBYTE              ;写命令字节
        LCALL  ACKN               ;收应答
        MOV    A,    SADDL
        LCALL  WBYTE              ;写2416的地址
        LCALL  ACKN               ;收应答
WD24:
        MOV    A,    @R1
        INC    R1
        LCALL  WBYTE
        LCALL  ACKN
        DJNZ   R2,   WD24
        LCALL  STOP               ; I²C 结束信号
        RET
```



- ;;----------
- ;;名称: R24C
- ;;功能: READ DATA FROM 24C16
- ;;输入: SADD,R1,R2
- ;;输出: (无)
- ;;----------
- ;;调用: START,WBYTE,ACKN,RBYTE,STOP
- ;;----------
- ;;影响: A,R1,R2
- ;;堆栈: 4
- ;;----------
- ;;参数说明:
- ;; SADD:24C16 ADDRESS
- ;; R2:WRITE NUMBER
- ;; R1:RAM ADDRESS POINTER
- ;;----------
- ;;注意:
- ;;----------

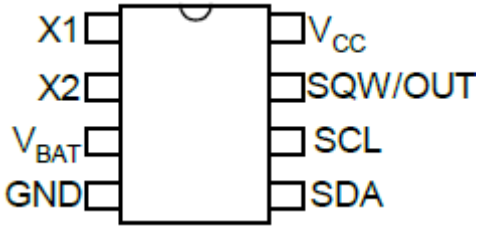


```
R24C:
        LCALL   START                   ; I²C 起始信号
        MOV   A,   SADDH
        RL   A
        ORL     A,      #0A0H
        LCALL   WBYTE                   ;写命令字节
        LCALL   ACKN                    ;收应答
        MOV     A,      SADDL
        LCALL   WBYTE                   ;地址字节
        LCALL   ACKN                    ;收应答
        LCALL   START
        MOV   A,   SADDH
        RL   A
        ORL     A,      #0A1H
        LCALL   WBYTE                   ;读命令字节
        LCALL   ACKN                    ;收应答
RC24:
        LCALL  RBYTE                    ;读2416
        MOV     @R1,A
        INC     R1
        DJNZ    R2,RCX
        LCALL   STOP                    ; I²C 结束信号
        RET
RCX:
        LCALL   ACKS                    ;发应答
        SJMP    RC24
```

I²C总线实时时钟 DS1307

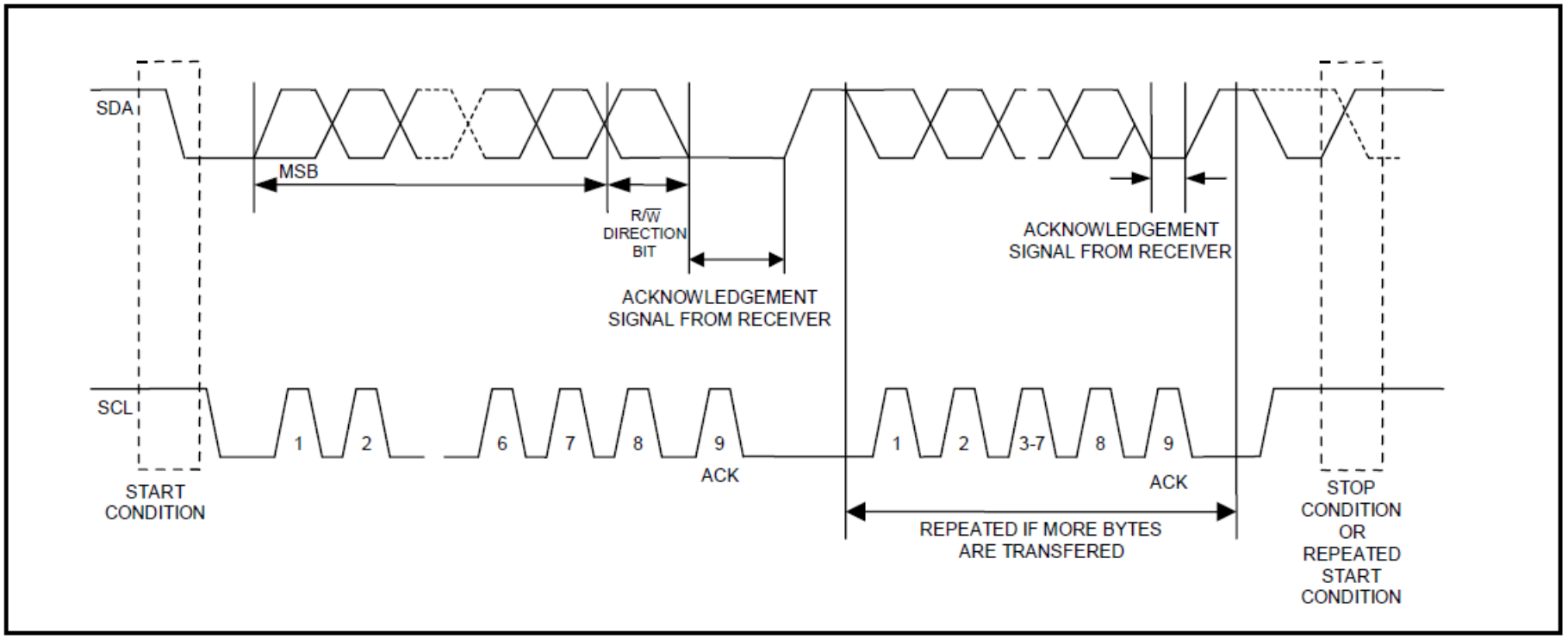
FEATURES

- Real-Time Clock (RTC) Counts Seconds, Minutes, Hours, Date of the Month, Month, Day of the week, and Year with Leap-Year Compensation Valid Up to 2100
- 56-Byte, Battery-Backed, General-Purpose RAM with Unlimited Writes
- I^2C Serial Interface
- Programmable Square-Wave Output Signal
- Automatic Power-Fail Detect and Switch Circuitry
- Consumes Less than 500nA in Battery-Backup Mode with Oscillator Running
- Optional Industrial Temperature Range: -40°C to +85°C
- Available in 8-Pin Plastic DIP or SO
- Underwriters Laboratories (UL) Recognized



I²C总线实时时钟 DS1307

Data Transfer on I²C Serial Bus



不同芯片的寻址

Figure 4. Data Write—Slave Receiver Mode

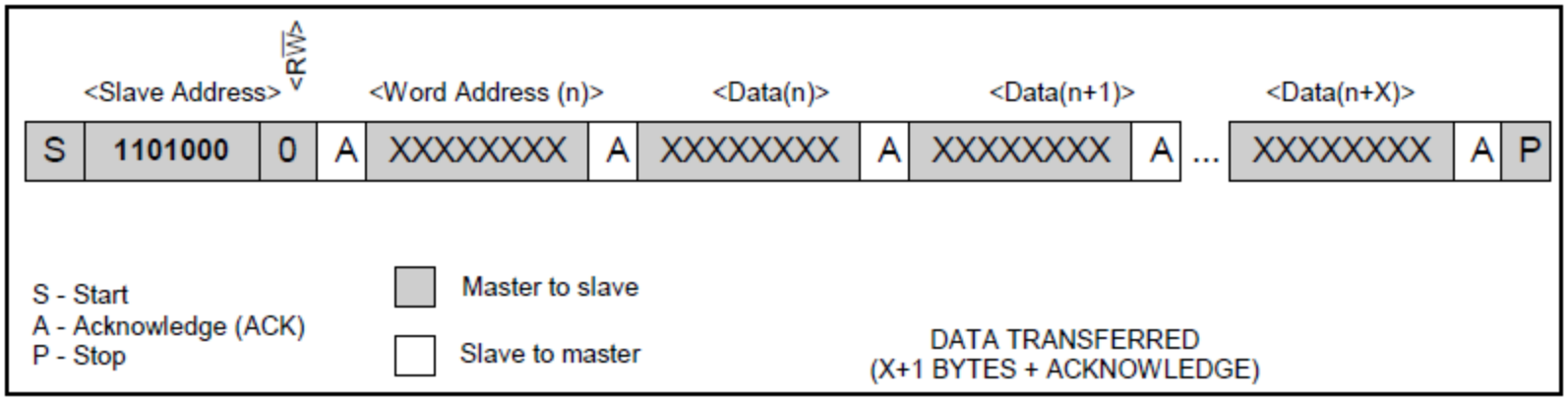
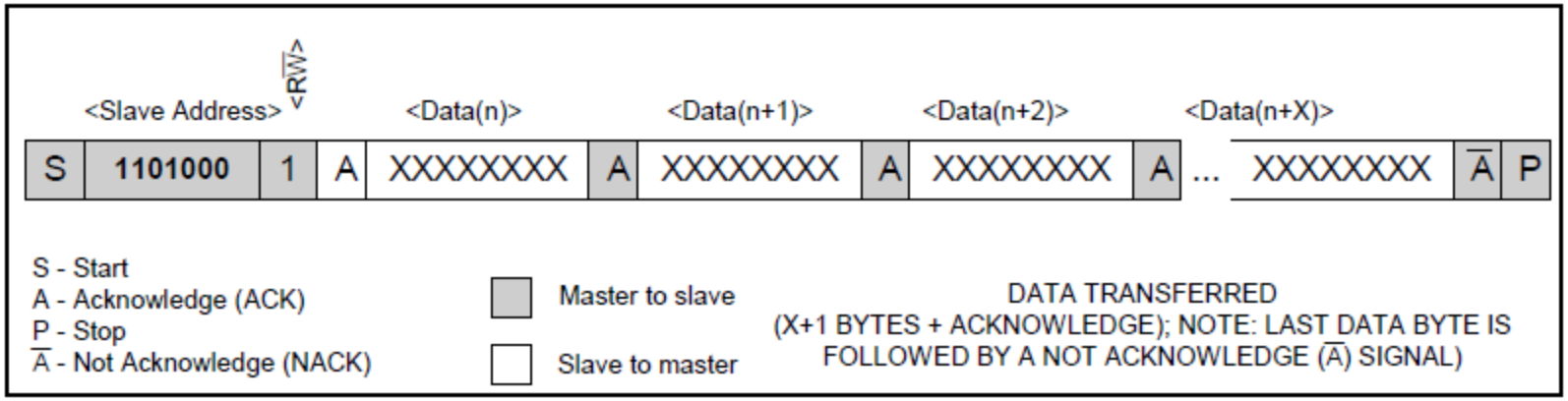
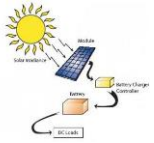


Figure 5. Data Read—Slave Transmitter Mode





Cortex-M4的I²C外设

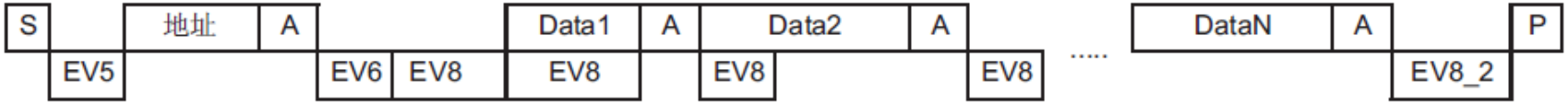
I²C外设特性

- 并行总线/I²C 协议转换器
- 多主模式功能：同一接口既可用作主模式也可用作从模式
- I²C 主模式特性： — 时钟生成 — 起始位和停止位生成
- I²C 从模式特性： — 可编程 I²C 地址检测 — 双寻址模式，可对 2 个从地址应答 — 停止位检测
- 7 位/10 位寻址以及广播呼叫的生成和检测
- 支持不同的通信速度： — 标准速度（高达 100 kHz） — 快速速度（高达 400 kHz）
- 状态标志： — 发送/接收模式标志 — 字节传输结束标志 — I²C 忙碌标志

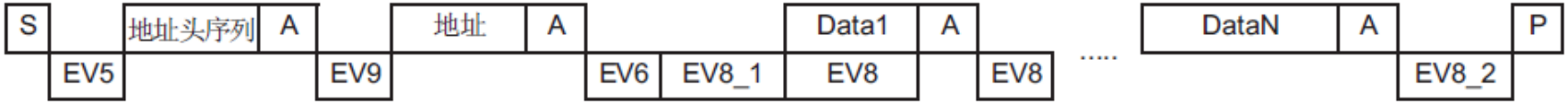
Cortex-M4的I²C外设

I²C外设做为主机的时序

7 位主发送器



10 位主发送器

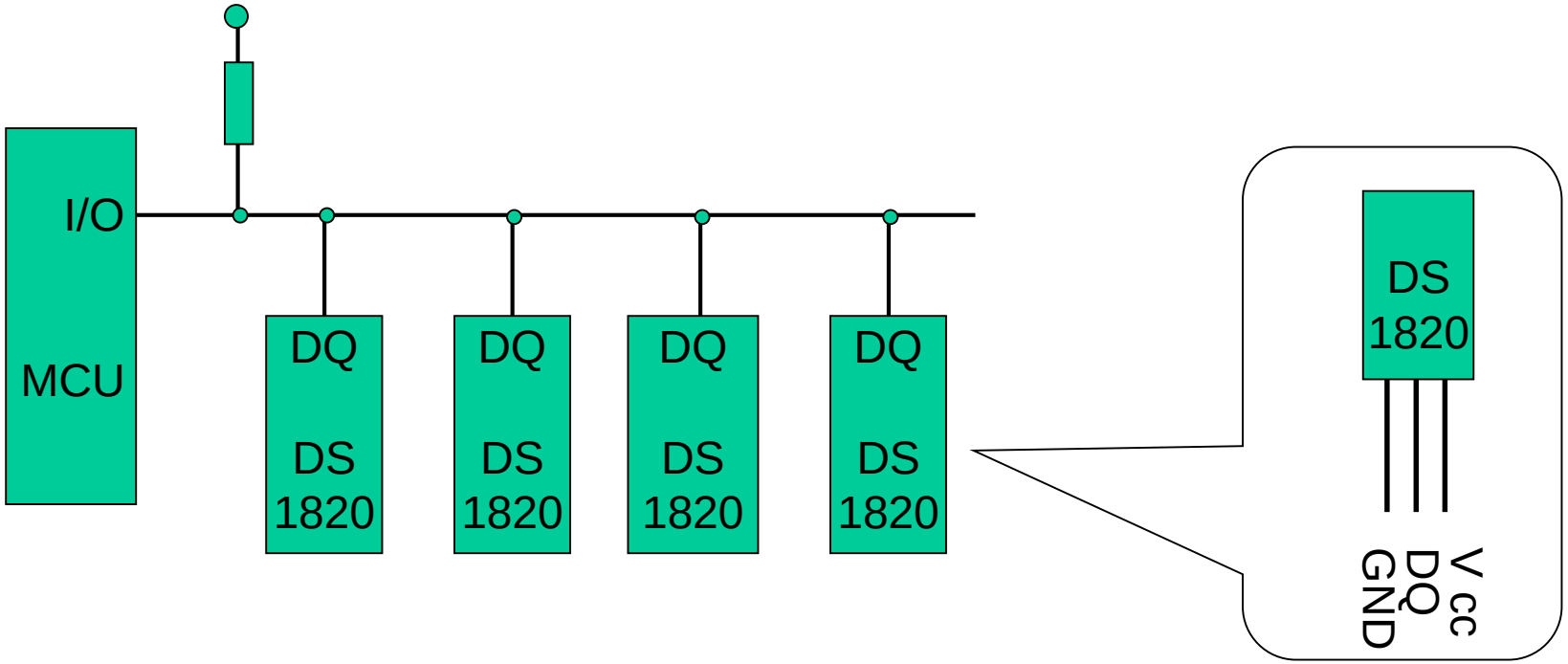


图注： S= 起始位，Sr = 重复起始位，P= 停止位，A= 应答，
EVx= 事件（如果 ITEVFEN = 1，则出现中断）

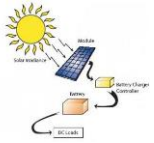
EV5：SB=1，通过先读取 SR1 寄存器再将地址写入 DR 寄存器来清零。
EV6：ADDR=1，通过先读取 SR1 寄存器再读取 SR2 寄存器来清零。
EV8_1：TxE=1，移位寄存器为空，数据寄存器为空，在 DR 中写入 Data1。
EV8：TxE=1，移位寄存器非空，数据寄存器为空，该位通过对 DR 寄存器执行写操作清零。
EV8_2：TxE=1，BTF = 1，程序停止请求。TxE 和 BTF 由硬件通过停止条件清零。
EV9：ADD10=1，通过先读取 SR1 寄存器再写入 DR 寄存器来清零。

ai18210

（四）单总线（1-wire）接口扩展系统



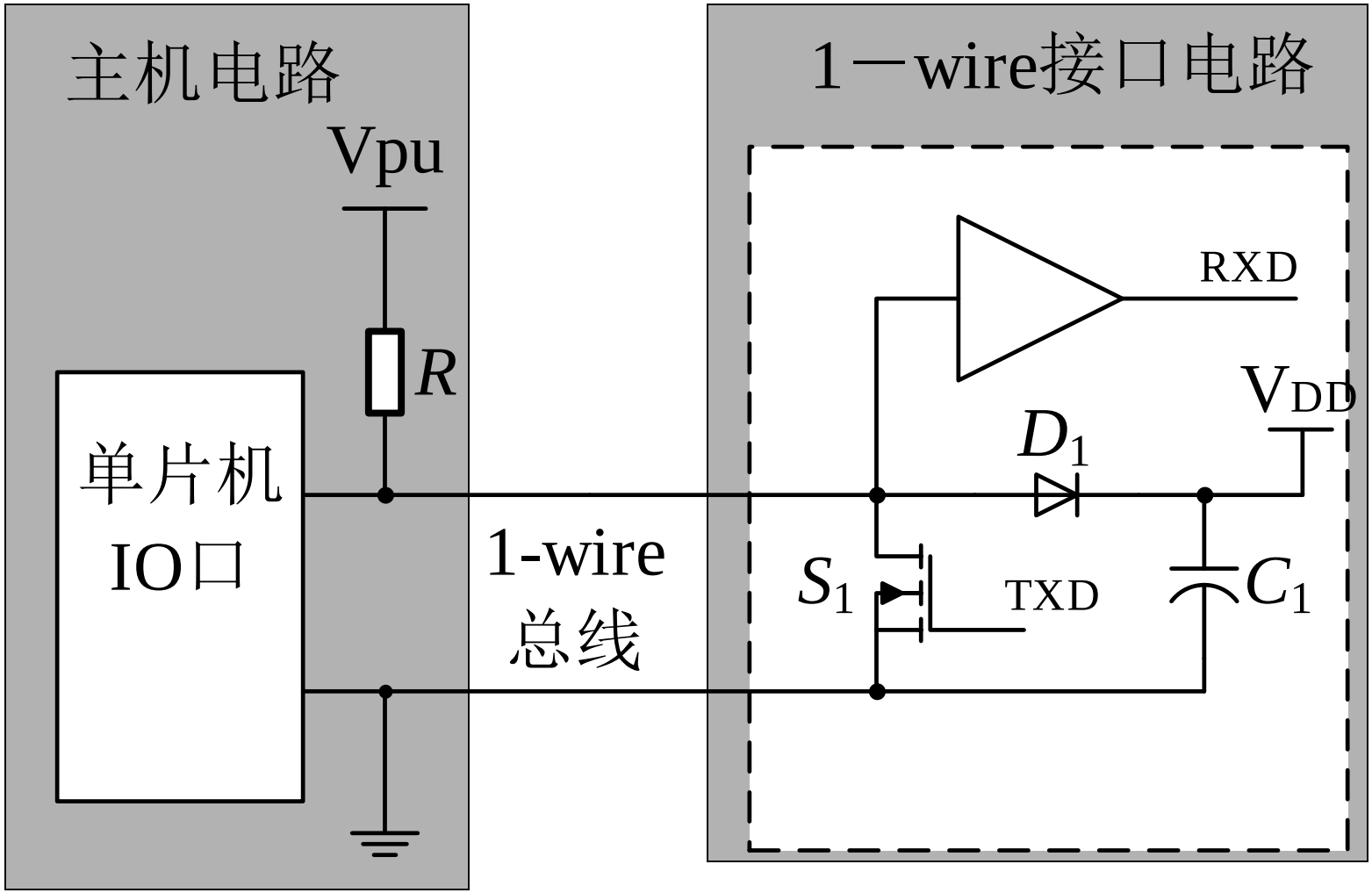
- DS18B20为美国DALLAS公司生产的温度计集成电路



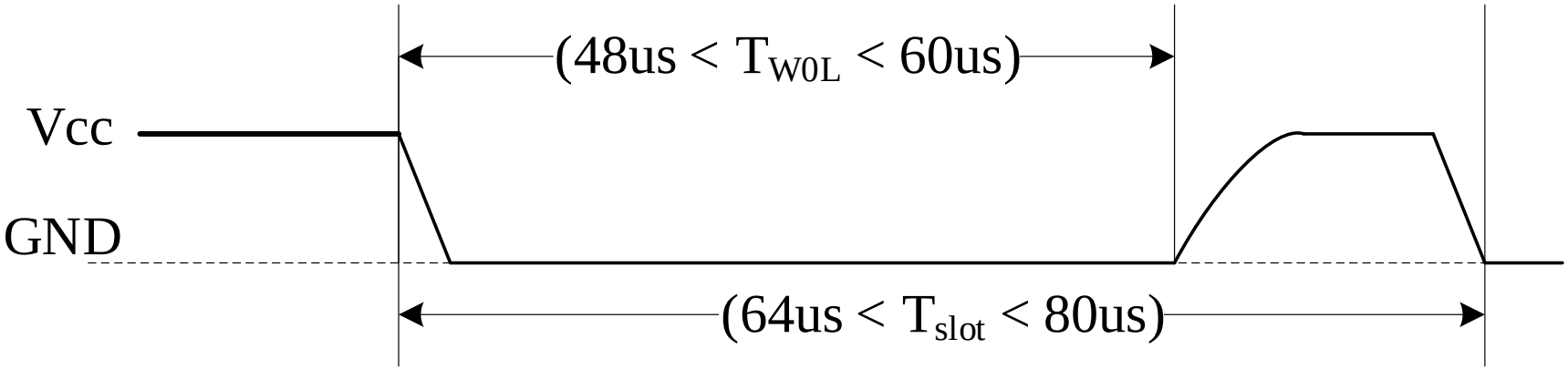
单总线接口工作原理

- 单总线（1-Wire bus）是美国DALLAS公司推出的外围串口扩展总线。
- 总线使用一根数据线DQ（输入/输出），总线上所有的器件都挂在DQ上。
- DALLAS公司生产的所有该总线的器件在其内部都有一个64位的ROM，厂家在出厂时使用激光烧写编码（十进制的产片序列号）作为每一个器件唯一的地址，以确保挂在总线上可以唯一确定并访问。
- DQ为漏极开路结构，故外部必须使用上拉电阻。
- 单总线器件具有低功耗特征，并具有电源存储电路，可以从DQ线上获取芯片的工作电源（当然也可以单独使用电源供电）。

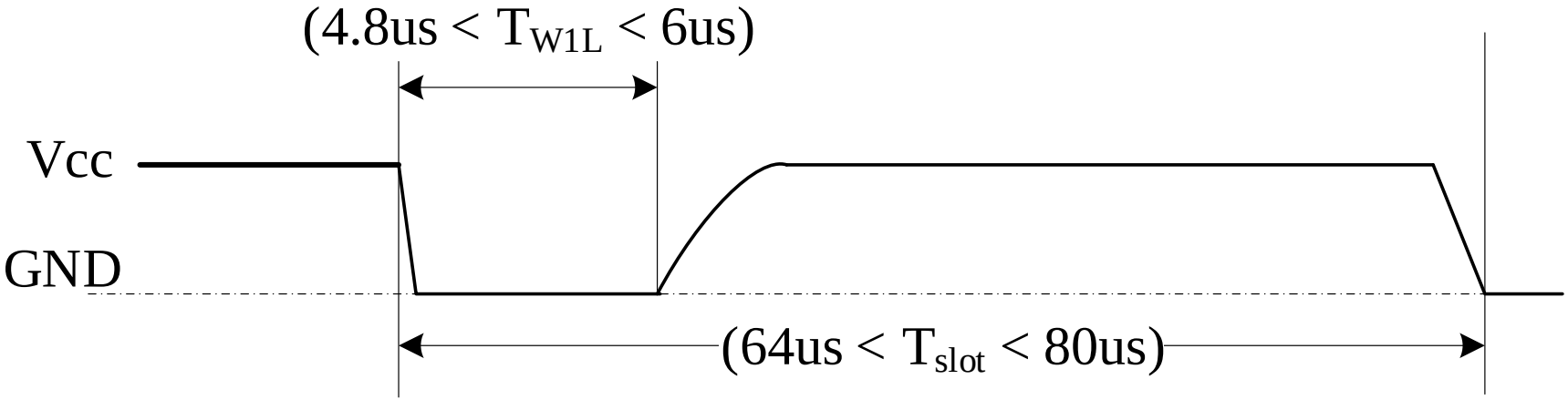
1—wire 总线供电原理



写 “1”时序和写 “0”时序

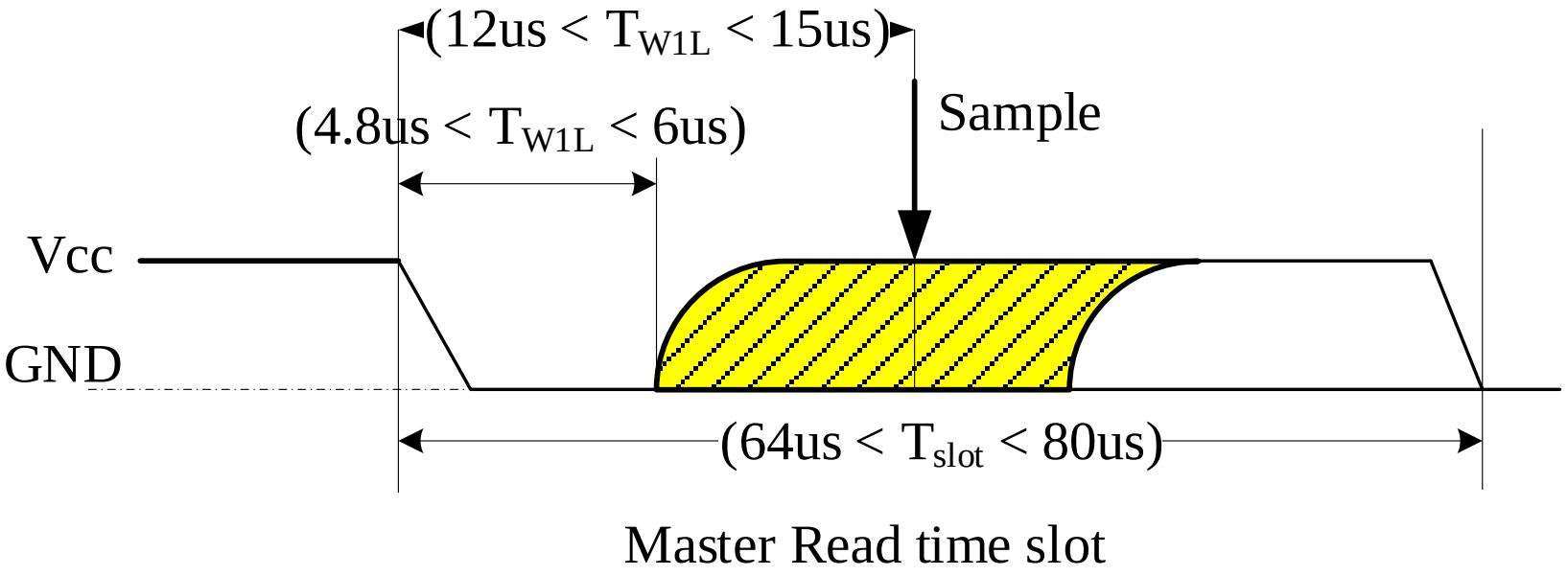


Write “0” time slot

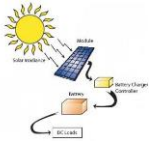


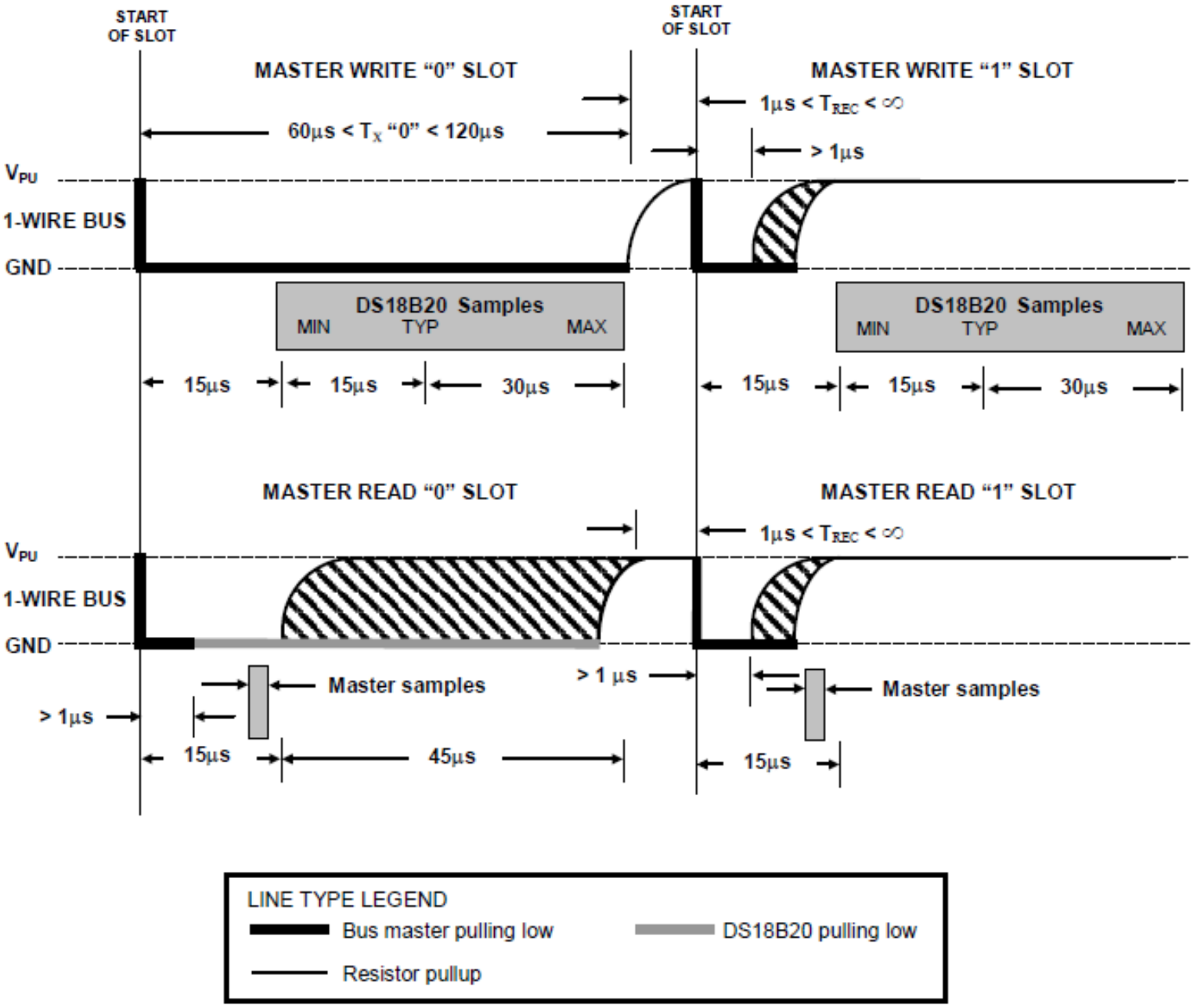
Write “1” time slot

主机读取数据时序

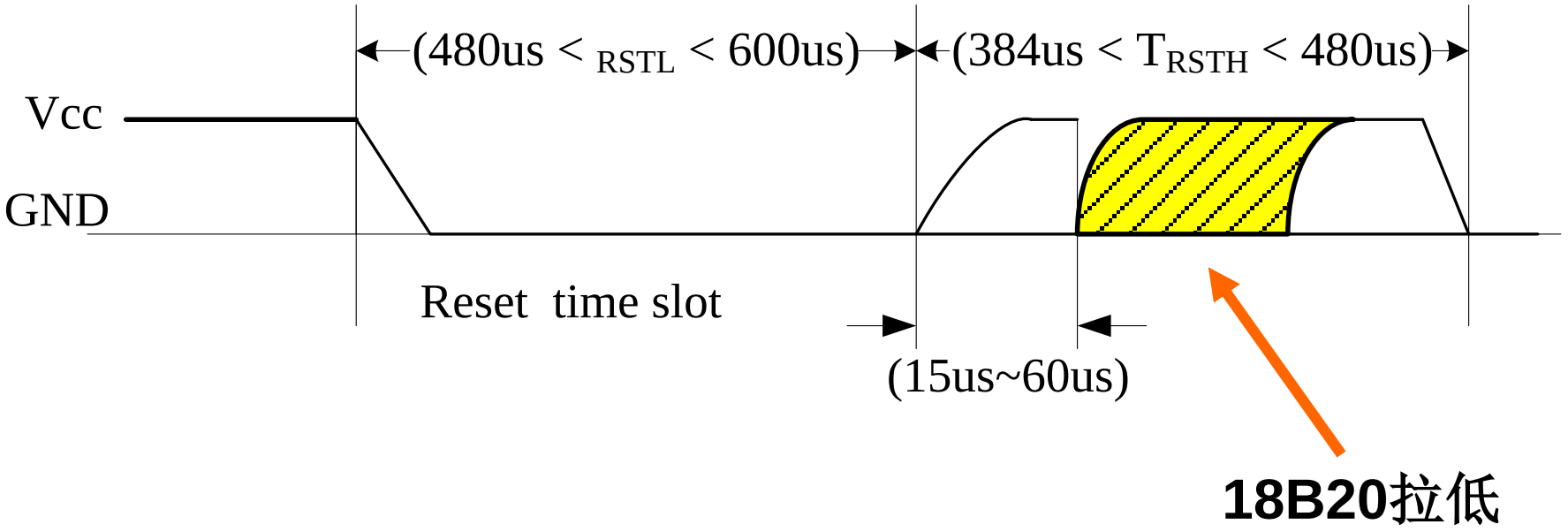


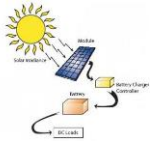
Master Read time slot




START OF SLOT
START OF SLOT
MASTER WRITE "0" SLOT
MASTER WRITE "1" SLOT
60µs < T_X "0" < 120µs
1µs < T_REC < ∞
> 1µs
V_PU
1-WIRE BUS
GND
DS18B20 Samples
MIN TYP MAX
15µs 15µs 30µs
15µs 15µs 30µs
MASTER READ "0" SLOT
MASTER READ "1" SLOT
1µs < T_REC < ∞
V_PU
1-WIRE BUS
GND
Master samples
> 1µs
> 1 µs
Master samples
15µs 45µs
15µs
LINE TYPE LEGEND
Bus master pulling low
DS18B20 pulling low
Resistor pullup

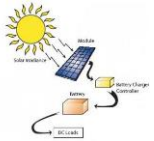
复位时序


(480us < RSTL < 600us)
(384us < TRSTH < 480us)
Vcc
GND
Reset time slot
(15us~60us)
18B20拉低



写一个字节（输入：A）

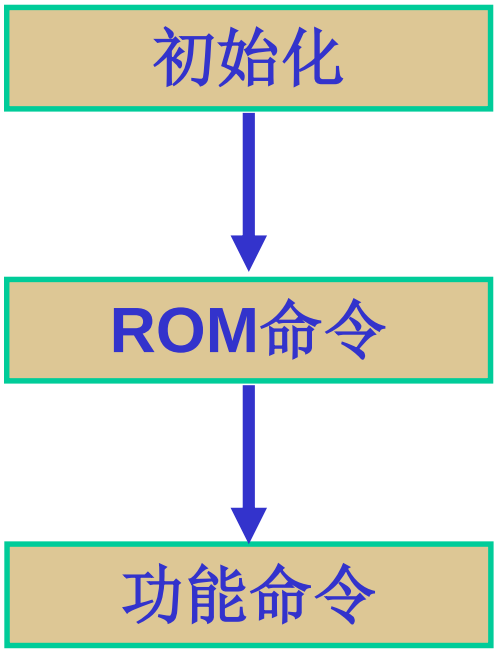
```
WRITE_BYTE:
        MOV R2,#8
        CLR  C
WR1:    CLR data_line
        MOV R3,#07
        DJNZ R3,$                ;总线复位保持16us以上
        RRC  A
;把一个字节DATA分成8个BIT循环移至C
        MOV data_line,C          ;写入一个BIT
        MOV R3,#3CH
        DJNZ R3,$                ;等待100us
        SETB data_line           ;重新释放总线
        NOP
        DJNZ R2,WR1              ;写入下一个bit
        SETB data_line
        RET
```



读一个字节数据（输出：A）

```
READ_BYTE:
        MOV     R0,#8
READ_LOOP1:
        CLR data_line       ;读前总线保持为低
        NOP
        NOP
        NOP
        SETB data_line      ;开始读总线释放
        MOV R3,#09           ;延时18微妙
        DJNZ R3,$
        MOV C,data_line      ;读得一个BIT
        MOV R3,#3CH
        DJNZ R3,$            ;等待100微妙
        RRC A                ;把读得的位值环移给A
        DJNZ R0,READ_LOOP1      ;读下一个BIT
        RET
```

18B20操作流程

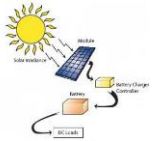


从机准备就绪

搜索设备地址

启动温度转换

读暂存器等



ROM命令

- SEARCH ROM（0F0H）
- READ ROM（33H）
- MATCH ROM（55H）
- SKIP ROM（0CCH）
- ALARM SEARCH（0ECH）

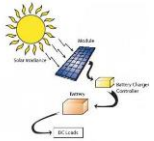


Table 3. DS18B20 Function Command Set

COMMAND	DESCRIPTION	PROTOCOL	1-Wire BUS ACTIVITYAFTER COMMAND IS ISSUED	NOTES
TEMPERATURE CONVERSION COMMANDS				
Convert T	Initiates temperature conversion.	44h	DS18B20 transmits conversion status to master (not applicable for parasite-powered DS18B20s).	1
MEMORY COMMANDS				
Read Scratchpad	Reads the entire scratchpad including the CRC byte.	BEh	DS18B20 transmits up to 9 data bytes to master.	2
Write Scratchpad	Writes data into scratchpad bytes 2, 3, and 4 (T_H, T_L, and configuration registers).	4Eh	Master transmits 3 data bytes to DS18B20.	3
Copy Scratchpad	Copies T_H, T_L, and configuration register data from the scratchpad to EEPROM.	48h	None	1
Recall E^2	Recalls T_H, T_L, and configuration register data from EEPROM to the scratchpad.	B8h	DS18B20 transmits recall status to master.	
Read Power Supply	Signals DS18B20 power supply mode to the master.	B4h	DS18B20 transmits supply status to master.	

Figure 7. DS18B20 Memory Map

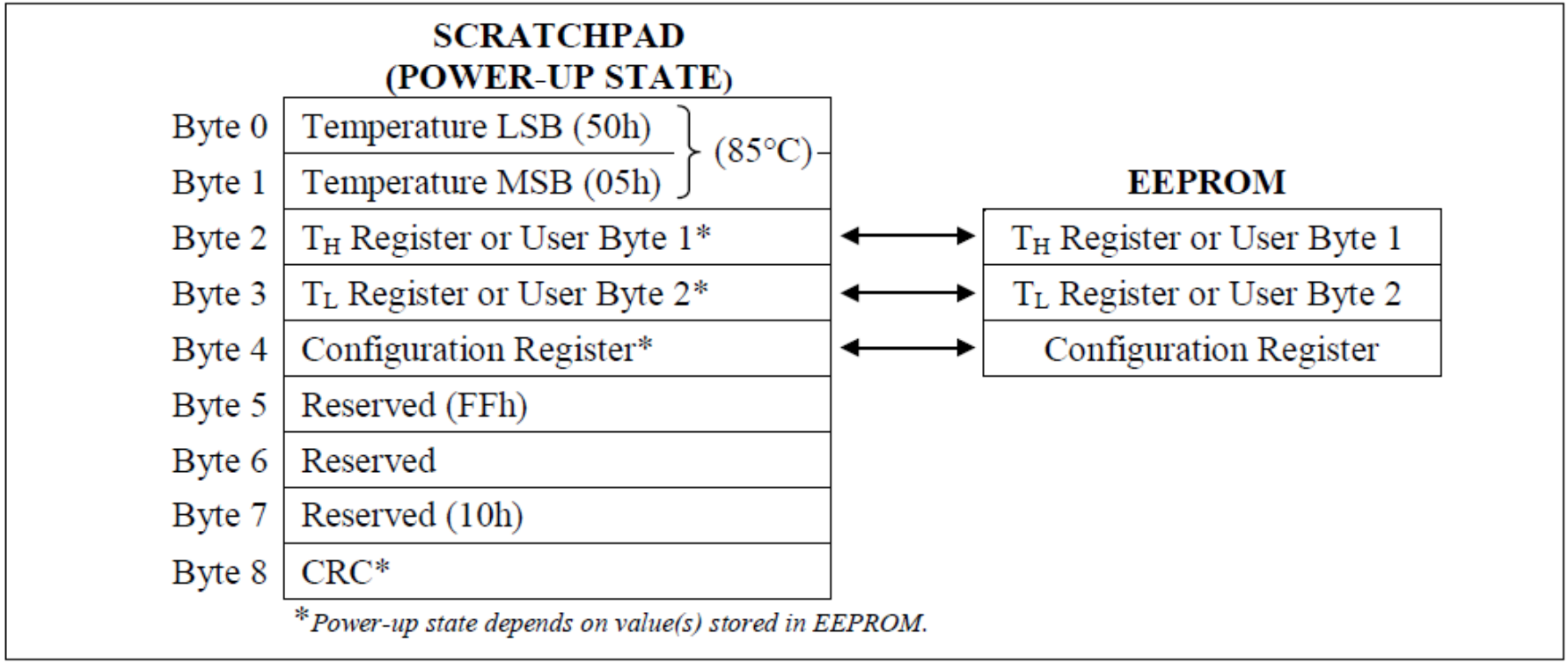


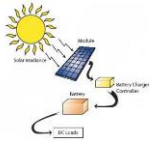
Figure 2. Temperature Register Format

	BIT 7	BIT 6	BIT 5	BIT 4	BIT 3	BIT 2	BIT 1	BIT 0
LS BYTE	2^3	2^2	2^1	2^0	2^{-1}	2^{-2}	2^{-3}	2^{-4}
	BIT 15	**BIT 14**	**BIT 13**	**BIT 12**	**BIT 11**	**BIT 10**	**BIT 9**	**BIT 8**
MS BYTE	S	S	S	S	S	2^6	2^5	2^4

S = SIGN

Table 1. Temperature/Data Relationship

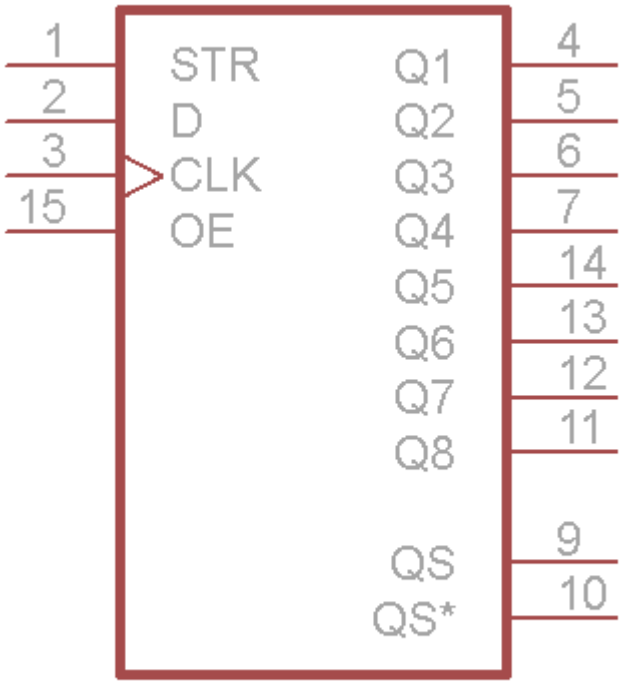
TEMPERATURE (°C)	DIGITAL OUTPUT (BINARY)	DIGITAL OUTPUT (HEX)
+125	0000 0111 1101 0000	07D0h
+85*	0000 0101 0101 0000	0550h
+25.0625	0000 0001 1001 0001	0191h
+10.125	0000 0000 1010 0010	00A2h
+0.5	0000 0000 0000 1000	0008h
0	0000 0000 0000 0000	0000h
-0.5	1111 1111 1111 1000	FFF8h
-10.125	1111 1111 0101 1110	FF5Eh
-25.0625	1111 1110 0110 1111	FE6Fh
-55	1111 1100 1001 0000	FC90h



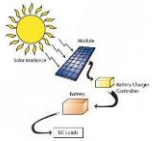
```
;;主程序
        LCALL  RESET
        MOV   A,#0CCH       ; 跳过ROM匹配
        LCALL  WRITE_BYTE
        MOV   A,#44H        ; 发出温度转换命令
        LCALL  WRITE_BYTE
        LCALL  RESET
        MOV   A,#0CCH       ; 跳过ROM匹配
        LCALL  WRITE_BYTE
        MOV   A,#0BEH       ; 发出读温度命令
        LCALL  WRITE_BYTE
        LCALL  READ_BYTE
        MOV     TMPL,A      ;温度低位
        LCALL  READ_BYTE
        MOV     TMPH,A      ;温度高位
       SJMP    $
```

作业

1.采用两片CD4094扩展16个输出口，画出该芯片与89C51的连线图（ 89C51 采用P1.0，P1.1，P1.2 口），并编写程序。



CD4094



END

THANK YOU!